

Ecommerce Application

Architecture API Reference and Interview Guide

Prepared for the project developer

Based on the repository at D:\spring projects\ecommerce

Source review date 15 September 2026

This handbook explains how the implemented application works, how requests cross its services, and why its validation, authorization, concurrency and recovery conditions exist. It includes concrete request and response examples for all application routes in 63 reference entries covering 64 handler methods, compatibility aliases, internal contracts and 60 project-specific interview answers.

The application uses local database transactions and durable recovery to coordinate commerce operations across services. Its most important interview topic is how checkout remains safe when HTTP calls, payment confirmations or event delivery fail at different times.

Examples are illustrative contracts derived from source code, not captured production traffic. IDs, customer information, tokens, timestamps and provider references are fictional. Operational and security gaps are called out where the source does not support a stronger claim.

Guide contents

Understanding the application

Architecture and service ownership

Control flow from registration to delivery

Reading states and data contracts

Complete application API reference

Infrastructure and provider interfaces

Conditions and why they exist

Operations testing and limitations

Interview questions and project answers

Source map and practical walkthrough

Use the Word Navigation pane to jump to any chapter, API operation or interview question. API sections identify access requirements, request bodies, success structures, important conditions, expected errors and the implementing classes. Empty bodies are stated explicitly. Framework generated error details can vary by Spring version and security filter path.

Understanding the application

What the system does

This application implements an ecommerce journey from customer registration and product browsing to shopping carts, checkout, payment reconciliation, fulfillment, returns and notifications. The customer storefront and administrator dashboard are static HTML, CSS and JavaScript served by the API gateway. The backend is a Maven multi-module Spring application with eight independently runnable services. Five business services own persistent domain data; three infrastructure services provide routing, discovery and centralized configuration.

The central design is a durable checkout coordinator in order-service. MySQL transactions protect each service's local state, while persisted progress, idempotent remote operations and scheduled recovery bridge service boundaries. This avoids claiming that a single `@Transactional` annotation can make MySQL, another HTTP service and Razorpay one atomic transaction.

The guide is for the project developer and interview preparation. Read the architecture and control flow first, use the API reference while exercising endpoints, and use the condition explanations and interview answers to explain both the implementation and its tradeoffs. Example names, addresses, IDs, tokens and provider references are illustrative. Replace them with values returned by a running instance; no sample token or signature is a usable credential.

Capabilities and boundaries

Implemented capabilities include account authentication and settings, product search and images, stock reservation, cart operations, COD and Razorpay test checkout, coupons, order history, fulfillment tracking fields, full-order returns, verified-purchase reviews, wishlists and a Kafka-backed in-app inbox. Reviews, coupons and wishlists live in order-service; inventory lives in product-service. There is no separate payment, review or shipping microservice in this repository.

Razorpay integration deliberately accepts test keys, not live keys. Carrier and tracking fields are manually supplied by an administrator; there is no courier API or return-label generator. COD refunds are externally paid and recorded by an admin reference. Returns cover a whole order within 30 days. Product images are HTTPS URLs rather than file uploads. Preferences do not implement real email, SMS or WhatsApp delivery. These are meaningful scope boundaries, not features to claim in an interview.

Technology choices visible in the repository

The modules target Java 21, use Spring Boot 3.5.x and Spring Cloud 2023.0.x, Spring MVC for business APIs and the WebFlux gateway starter for routing. The POMs are not on one identical patch version: the gateway uses Boot 3.5.15 while several business services use 3.5.16. Spring Data JPA and Hibernate handle persistence, MySQL is the application database, and tests use H2. JWT signs and parses access tokens; Spring Security enforces route and method authorization.

Kafka carries user and order events. Redis is an optional cache for product-by-ID reads. Resilience4j circuit breakers are attached to RestClient requests in cart-service and order-service. Docker Compose and Kubernetes manifests provide deployment scaffolding. Actuator, structured logs and optional OpenTelemetry export to Tempo provide operational signals. These are descriptions of the checked-in configuration, not a claim that a production cluster has been deployed.

Architecture and service ownership

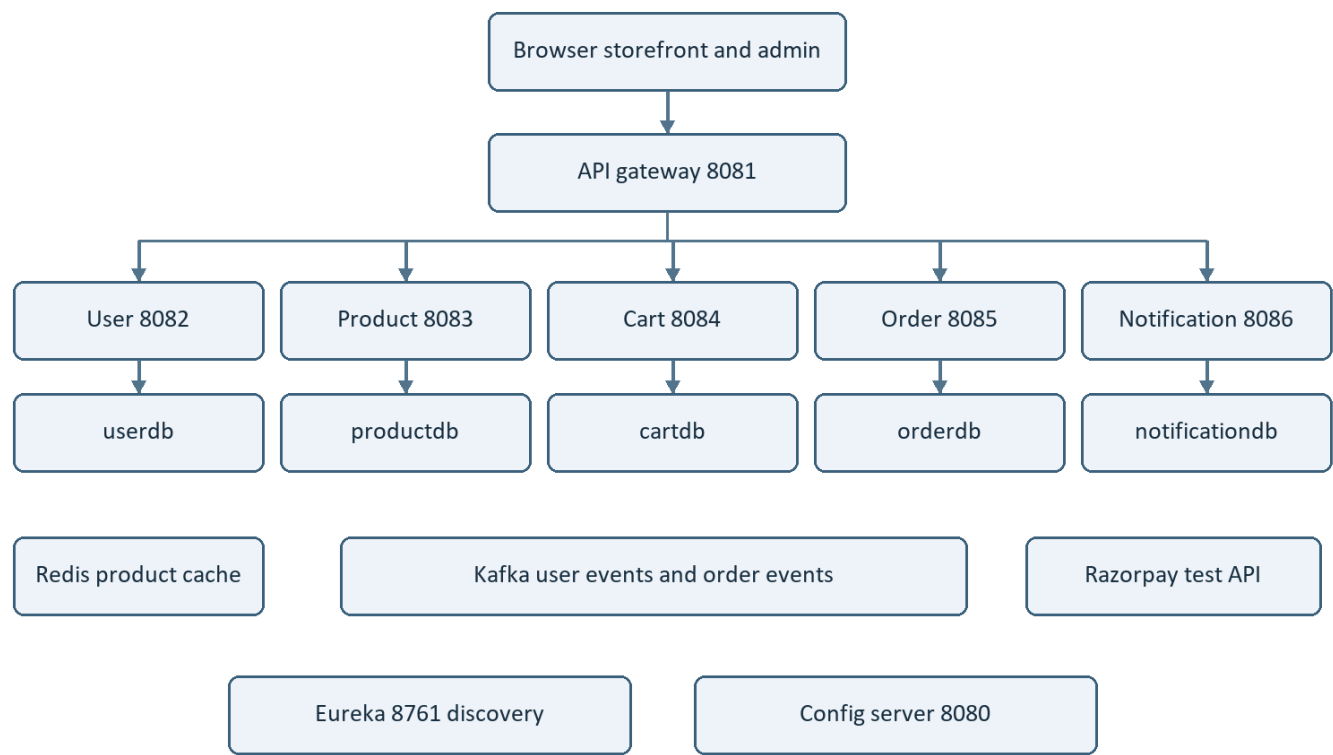


Figure 1. Each business service owns its database. Order-service calls user, cart, product and Razorpay synchronously; order events reach notification-service through Kafka, while user events are published to a separate topic. Redis supports product reads. Discovery and configuration support the runtime rather than owning commerce data.

Service	Port	Responsibility
service-registry	8761	Eureka service discovery
config-server	8080	Central configuration source
api-gateway	8081	Routing and static browser UI
user-service	8082	Accounts authentication addresses preferences
product-service	8083	Catalog stock reservations Redis cache
cart-service	8084	Versioned customer cart
order-service	8085	Checkout payments fulfillment coupons reviews wishlist returns
notification-service	8086	Kafka events and customer inbox

The request path

A browser loads `index.html`, `store.css` and `store.js` from `api-gateway` on port 8081. It makes relative `/api` requests to that same origin. Explicit gateway routes forward product calls to `product-service`, `user/account` calls to `user-service`, `cart` calls to `cart-service`, `order` and `commerce` calls to `order-service`, and `notifications` to `notification-service`. The gateway forwards headers and bodies; JWT enforcement resides in the destination services.

Within a typical business service, the security filter establishes Authentication, the controller validates input and converts HTTP parameters, the service enforces business rules, the repository executes persistence operations and DTOs or selected entities become JSON. Authentication is not sufficient authorization: ownership and `ROLE_ADMIN` checks must occur at the relevant boundary as well.

Gateway routes normally use `lb://service-name` and Eureka to resolve an instance. In contrast, `ServiceClients` and `ProductClient` use configured base URLs through ordinary `RestClient`. They are not automatically service-discovery clients simply because Eureka is on the classpath. Container properties supply service DNS addresses; IDE defaults use localhost ports.

The eight services

`service-registry` is the Eureka registry on port 8761. It stores the current registration view used by discovery clients. It has no customer business database and does not perform checkout decisions. Registry availability affects discovery, but a cached registry and configured direct URLs are distinct mechanisms.

`config-server` normally runs on port 8080 and exposes Spring Cloud Config configuration from a Git backend. The native profile reads `ecommerce-config-repo` from the workspace. IDE startup uses Config Server imports. Container configuration disables remote Config Server loading and imports bundled application-container.properties, so do not assume Config Server is in every runtime request path.

`api-gateway` runs on port 8081. It serves the storefront and routes requests to backend services. It has no commerce database. The container profile disables `discovery-locator` routes and uses explicit API routes; the workspace `config-repo` profile enables the locator. That difference matters when reasoning about which paths can be exposed.

`user-service` runs on port 8082 and owns `userdb`. It stores users, roles, refresh tokens, verification/reset tokens, addresses, preferences and login history. It issues JWTs and `user-events`. `Product-service` runs on 8083 and owns `productdb`, including catalog records and inventory reservations. It provides public browsing, admin writes and `service-key-only` inventory endpoints.

`cart-service` runs on port 8084 and owns `cartdb`. It stores per-user cart quantities and versions, calculates estimates using current product information and exposes internal versioned checkout snapshots. `Order-service` runs on port 8085 and owns `orderdb`. It stores purchase orders, outbox records, coupons, reviews and wishlists, coordinates checkout, invokes Razorpay and controls return/refund state.

`notification-service` runs on port 8086 and owns `notificationdb`. It consumes `order-events`, deduplicates by `eventId` and exposes a customer's inbox. It does not consume `user-events` in the present implementation. Consequently, an OTP event from `user-service` does not become an email or notification merely because `notification-service` exists.

Database ownership and important entities

User relationships include users to roles through `user_roles`, users to many addresses and token/history rows, and a one-to-one preferences association. BCrypt hashes are ignored by JSON serialization and `UserResponse`

selects a limited public profile. Address inherits auditing timestamps and ignores its User relationship to avoid recursive serialization and disclosure.

Product is versioned and has a normalized unique SKU, name, description, decimal price, available stock, category, active flag, image URL and timestamps. StockReservation uses the order UUID as its ID and stores reservation state and a line collection. Each line snapshots product ID, name, price and quantity. This separates a product's current mutable price from a purchased price.

Cart uses the user ID as its primary key and has @Version plus a product-to-quantity element collection. It is not a price ledger. PurchaseOrder has a string UUID, numeric user ID, address ID, hashed checkout key, cart version, separate order/payment/refund states, copied JSON address/cart/items, monetary snapshots, provider references, progress flags and timestamps. Coupon has a code primary key, rule values, expiry, usage limit/counter and version. Review and WishlistEntry enforce unique user/product pairs. OutboxEvent stores a durable event payload with publication status.

Foreign keys cannot span independent service databases in the normal deployment model. References such as PurchaseOrder.userId or Review.productId are application-level identifiers, not JPA associations into another service. Within orderdb, some references are scalar values rather than declared foreign keys. This reduces coupling but puts consistency responsibility on service checks, snapshots and recovery.

Why these boundaries are useful and where they are costly

Separate ownership keeps inventory rules in product-service and order state in order-service. A cart can fail or be edited without directly corrupting order price history. The gateway provides one client origin and route surface. The approach also introduces network latency, partial failures, duplicated identity logic and operational overhead: eight JVM services plus databases, Kafka and Redis are substantial for a learning application.

For a small team or low-traffic product, a modular monolith could provide the same domain separation with simpler transactions and deployment. A strong interview answer explains why this repository demonstrates distributed-system techniques without insisting that microservices are always the most economical production starting point.

Control flow from registration to delivery

Registration and login

Registration passes through user-service validation, normalizes email, checks unique email/phone, hashes the password with BCrypt and saves a `ROLE_USER` customer. It creates default preferences and a verification OTP, then publishes user events. Login finds the email, checks lock and account status and compares the password. On failure it increments attempts and persists history; after five failures the account is locked for 30 minutes. The login transaction is configured not to roll back `ApiException`, because rolling back would erase the very failure counter used for logout.

On successful login the service records last login/count/history and returns an access JWT and a persisted opaque refresh token. The browser stores the auth response in `sessionStorage`. Expired access requests can trigger a single shared refresh promise in `store.js`, preventing multiple components in one tab from refreshing at once. The backend revokes the old refresh token and creates a new one. This frontend guard does not solve concurrent refresh races between different devices or processes.

Browsing and the cart

`ProductSearch` builds a JPA Specification from optional query, category, price and in-stock filters. Only active products reach the public list. Public detail routes also hide inactive products, while an admin can inspect them. Lists query MySQL directly; `product-by-ID` reads first ask MySQL for the entity version and may then reuse a Redis JSON response keyed by ID and version.

Adding a cart line reads current product state and checks total desired quantity. It stores quantities and updates the cart version; it does not deduct inventory. A cart GET reprices every line against the current catalog. Missing or hidden products remain visible as unavailable lines so they can be removed. This means a cart total is an estimate and can change before checkout. Actual stock allocation happens only during reservation.

The synchronous checkout entry point

`OrderController` reads `Authentication.userId`, `Authorization` and `Idempotency-Key` and validates Checkout. `OrderService` hashes the key with SHA-256 and looks for an existing order with that user/key pair. If one exists, its request identity must match address, method and normalized coupon; no new cart is fetched for that retry. This is why the retry can work after the original cart was consumed.

For a new attempt the service validates a coupon if present, requires Razorpay test configuration for online payment, fetches the user's addresses with the customer bearer token and selects the owned address, then fetches the internal cart snapshot with the service key. It rejects empty or malformed snapshots and more than 100 distinct lines. It saves a new `CREATING` order with a UUID, address/cart snapshots, cart version and deadline for online payment.

The database enforces uniqueness of both `(user_id, key_hash)` and `(user_id, cart_version)`. Concurrent first requests may race, but only one can persist that identity. A duplicate key loads the winner; a different key on the same cart version is a conflict. The order UUID is reused as the inventory reservation ID and provider receipt. The HTTP request then calls `advance` to try progressing the durable work.

Reservation and authoritative pricing

Within a local order transaction, `advance` locks the order and proceeds only when status is `CREATING`. It calls the internal reservation endpoint. `Product-service` returns an existing matching reservation or creates a new

reservation, locks product rows in ascending ID order, checks active stock and deducts all quantities atomically in productdb. It returns the stored line prices and names.

OrderService verifies the reservation ID, RESERVED state, exact number of lines, unique IDs, requested quantities, non-null names and positive prices. It saves the item JSON and calculates the authoritative subtotal using BigDecimal. CouponService locks a coupon in the same orderdb transaction, checks eligibility again and records discount/use. COD moves to CONFIRMED. Online checkout moves to RESERVED before provider order creation.

If stock cannot be allocated, a 422 response causes a durable FAILED order. If the coupon becomes invalid after stock was reserved, the service records CANCELLING with closureReason FAILED, releases stock through the same compensation machinery, then reaches FAILED. CouponRejectedException is excluded from automatic rollback in CouponService.apply so the outer transaction can retain that failure transition and recover safely.

Online payment creation and verification

The order commits paymentCreationAttempted before contacting Razorpay. A first attempt posts amount in paise, INR currency, receipt equal to local UUID and partial_payment=false. If the response is uncertain, a later attempt searches by receipt rather than blindly creating another provider order. A matching response must have the expected receipt, amount, currency and valid order ID before the local order reaches PENDING_PAYMENT.

The frontend obtains only the publishable test key ID and the checkout order details. It opens Razorpay Checkout and sends the callback fields to the verification API. The backend checks the signature over stored_provider_order_id + "|" + payment_id and independently reads provider order/payment records. A browser success callback alone is not authoritative. The captured payment must belong to the exact order and have the exact amount and INR currency; the provider order must be paid.

Verified captures move an awaiting order to CONFIRMED and PAID and produce outbox events. Signed webhooks and the recovery worker offer alternative paths if a browser callback is lost. A capture arriving after expiry/cancellation does not reopen the order or reserve stock again: it requests a refund. The system therefore separates payment success from entitlement to fulfill an already-closed order.

Cleanup and scheduled recovery

After advancement, cart consumption uses both user ID and captured cart version. Cart-service locks its cart and clears only a matching version. A cart changed during checkout is retained. The order tracks cartCleanupDone and cartCleared separately: the attempt can be complete even when the cart was deliberately not cleared.

OrderRecovery runs every 30 seconds by default and fetches at most 100 recoverable order IDs ordered by updatedAt. It retries CREATING, RESERVED, PENDING_PAYMENT, CANCELLING and RETURN_RECEIVING work, outstanding inventory commits/cart cleanup, online closed-order refunds and released-order coupon accounting. Outbox publication runs independently every five seconds in bounded batches.

advance is not a giant transaction. Individual steps use TransactionTemplate and persisted flags. Many RestClientException and ResponseStatusException failures are caught as recoverable dependencies, logged and revisited. A 202 order response communicates incomplete checkout; some administrative methods still return 200 with an intermediate state, so callers must inspect status. Recovery is eventual, not a guarantee that every error resolves without operator intervention.

Fulfillment and cancellation

An admin advances CONFIRMED to PROCESSING, then SHIPPED, then DELIVERED. Shipping requires carrier and tracking number; COD delivery requires cashCollected=true and changes paymentStatus to COLLECTED. An online order must already be PAID. Duplicate shipping updates must use identical tracking details.

Shipping commits the reservation in product-service. COMMITTED stock is not cancellable through the release operation. Cancellation before shipment records CANCELLING, releases RESERVED inventory and then finalizes CANCELLED or EXPIRED. A released tombstone is created even for an unknown reservation ID, preventing a late reservation request from allocating stock after the cancellation won the race. Coupon usage is released only after a failed/cancelled/expired state is finalized.

The fulfillment state can be committed before a subsequent remote inventory-commit call fails. The recoverable query includes pending commits. This is an example of why a failure response should not be interpreted as proof that no state changed: refresh and retry the same business transition, rather than issuing a different conflicting transition.

Returns and refunds

A customer requests a return only for an owned DELIVERED order within 30 days of deliveredAt. The request stores a reason and returnStatus REQUESTED without immediately changing inventory or money. An admin approves or rejects it with a note. At approved physical receipt, the admin chooses to restock all units or discard all units; partial returns are outside scope.

The service commits RETURN_RECEIVING and the restock choice before the remote call. Product-service makes the stock disposition idempotent using reservation state. A retry with the same decision does not restore stock twice; a different decision conflicts. After receipt, the order becomes RETURNED, returnStatus RECEIVED and an event is recorded.

For Razorpay, refund machinery uses the amount actually paid after discount and a durable refund attempt UUID. It sends X-Refund-Idempotency with an identical request across uncertain retries, validates provider refund ID/payment/amount/currency and tracks pending, processed or failed states. Only a confirmed failed provider refund permits a new attempt. An external partial refund triggers REVIEW_REQUIRED rather than pretending it is a normal full refund.

For COD, receipt marks MANUAL_REQUIRED. An administrator pays the customer outside this application and records a reference; that attestation marks the order REFUNDED. The application has no bank-transfer API. Returned orders do not release coupon usage, because the rule counts the completed purchase rather than allowing a return to recycle the promotion.

Events and notification delivery

OrderEvents requires an existing transaction through propagation MANDATORY. Each event increments the order's eventSequence and inserts an outbox row in the same transaction as the state transition. OutboxPublisher locks the oldest unpublished row, sends it to order-events using orderId as the key, waits up to ten seconds for acknowledgment, then records publishedAt.

If Kafka accepted the record but the database commit fails, the publisher can send it again. Therefore this is at-least-once delivery with consumer deduplication, not global exactly-once delivery. NotificationConsumer checks eventId, validates the envelope and saves an inbox entry with that ID. The listener retries persistence errors indefinitely every five seconds rather than silently discarding them. Unknown future types are ignored

intentionally. A permanently malformed record can therefore stall a partition; a dead-letter policy is a future operational improvement.

user-service takes a different approach: EventPublisher sends directly with optional KafkaTemplate and no outbox. Its payload includes type, userId, email and occurredAt. There is a database/event dual-write gap, and no OTP value is included. The reliability guarantees of the order outbox must not be attributed to the user event publisher.

Reading states and data contracts

Independent state dimensions

Order status describes business progress: CREATING, RESERVED, PENDING_PAYMENT, CONFIRMED, PROCESSING, SHIPPED, DELIVERED, CANCELLING, CANCELLED, EXPIRED, FAILED, RETURN_RECEIVING and RETURNED. Payment status describes money: UNPAID, PAID, COLLECTED, REFUND_PENDING, REFUND_FAILED or REFUNDED. Refund status includes NONE, REQUESTED, PENDING, PROCESSED, FAILED, REVIEW_REQUIRED and the COD-specific MANUAL_REQUIRED.

Return status is null internally until a request and is exposed as NONE in OrderView, then REQUESTED, APPROVED or REJECTED, RECEIVING and RECEIVED. Inventory reservation state is RESERVED, COMMITTED, RELEASED, RETURNED or RETURNED_DISCARDED. These dimensions are related but not interchangeable. For example a CANCELLED online order can still have REFUND_PENDING, and a DELIVERED COD order is COLLECTED rather than PAID.

Several persisted statuses are strings, while request enums restrict payment method, fulfillment target and return decisions. This simplifies schema changes but allows invalid values from manual database edits. There is no general endpoint to set any arbitrary order state. Transitions are enforced in service methods, not just by the UI buttons.

Money snapshots and nulls

Product and order amounts use decimal database columns and BigDecimal in Java. Product.price is in INR units; Razorpay amount is the exact integer `total.movePointRight(2).longValueExact()`. An order with subtotal 2000, discount 200 and totalPrice 1800 sends `amount=180000` to the provider and refunds that paid total.

Before reservation, items can be empty and totals null. checkout is present only when a provider order exists and status is PENDING_PAYMENT. failureReason describes durable setup failure; refund.failure describes refund failure. carrier, tracking and delivery timestamps are null until the relevant transition. Existing historical orders without new subtotal/discount fields are returned with subtotal falling back to total and discount to zero.

The coupon cap is $\min(\text{calculated_discount}, \max(\text{subtotal} - 1, 0))$. For subtotals at least INR 1 this leaves at least INR 1 payable; an item priced below INR 1 receives zero coupon discount and remains below INR 1. The source comment describes a one-rupee floor, but the formula does not increase a sub-rupee subtotal. This edge case should be checked against provider minimum amounts before a live integration.

API conventions and example interpretation

Use `http://localhost:8081` for explicitly routed customer/admin APIs. Internal examples instead use `product-service` on 8083 or `cart-service` on 8084 and `X-Service-Key`. User-service-only legacy aliases are documented separately; they are not explicit gateway routes. Requests with JSON bodies use `Content-Type: application/json`. Customer APIs use `Authorization: Bearer <ACCESS_TOKEN>`. Admin endpoints require a signed roles claim containing `ROLE_ADMIN`. The webhook uses `X-Razorpay-Signature` over exact bytes.

Samples below are structural examples for each handler, not HTTP captures. Server-generated IDs, timestamps, ordering and provider values vary. A Page response includes content and pagination metadata; exact Spring Data metadata serialization is framework-generated and is not a custom stable public DTO. Empty lists have `content=[]` and appropriate zero totals. A 204 response has no JSON body, not an empty JSON object. Code examples use null for absent fields.

The tables and API entries cover every explicit application controller handler plus path aliases, implicit HEAD behavior, service internals and a separate inventory of operational framework surfaces. Framework-owned Eureka, Actuator and Config Server APIs are not custom controller implementations; their available endpoints depend on profiles and dependency configuration. They are documented as operational surfaces instead of invented business APIs.

Error shapes differ between services

User-service `ApiException` becomes HTTP 400 with `{"message":"..."}`; integrity conflicts become 409. This means its missing/foreign address error is not the same 404 contract used by order-service. Its generic exception handler returns an opaque 500 message. Do not document a universal exception envelope that the code does not implement.

Product-service uses `ApiError` with timestamp, status, error, message, path and `fieldErrors` for its explicitly handled exceptions. Cart-service and order-service use Spring `ProblemDetail` for many business/dependency errors, with type, title, status, detail and usually instance. Framework validation, malformed JSON, authorization filters and `sendError` can take framework-specific response paths. Program against HTTP status and supported fields rather than assuming every error includes the same keys.

Complete application API reference

Default base URL is `http://localhost:8081` for explicit `/api` routes. Internal requests use the stated service port and `X-Service-Key`. Legacy aliases and `/hello` are service controller mappings; do not assume they are explicit public gateway routes. Authentication is enforced by each business service. JSON requests use `Content-Type: application/json`, and JSON responses normally use `application/json`. Replace concrete IDs in paths with returned values. Pagination examples show a typical Spring Page serialization; metadata ordering and framework details are not a stable custom DTO promise.

Authorization: Bearer `<ACCESS_TOKEN>` is required unless the entry states `Public`, a refresh token, a webhook signature or a service key. Admin tokens require `ROLE_ADMIN`. `Public` means authentication is not required; a malformed Authorization header may still be rejected by a service filter. A response body of null in this guide is never a replacement for an explicitly stated empty response body.

Common error structures

Errors are not globally standardized. User-service custom business failures return a message object. Product-service advice returns an `ApiError`. Cart and order business exceptions use `ProblemDetail` in relevant advice paths. Security entry points, request binding failures and framework validation can use different structures. Treat the endpoint error notes as status and condition guidance rather than assuming every failure shares the following body.

```
{"message": "The request conflicts with existing data"}
```

```
{
  "timestamp": "2026-09-15T10:00:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Product not found",
  "path": "/api/v1/products/999",
  "fieldErrors": {}
}
```

```
{
  "type": "about:blank",
  "title": "Conflict",
  "status": 409,
  "detail": "Cart changed concurrently. Reload the cart and retry.",
  "instance": "/api/cart/items"
}
```

User service

API 1 Register a customer

POST /api/users/register

Access: Public

Request body

```
{
  "name": "Asha Sharma",
  "email": "asha@example.com",
  "phone": "+919876543210",
  "password": "ExamplePass123!"
}
```

Success response example HTTP 201

```
{
  "id": 1,
  "name": "Asha Sharma",
  "email": "asha@example.com",
  "phone": "+919876543210",
  "gender": null,
  "dob": null,
  "profileImage": null,
  "memberSince": "2026-09-15T10:00:00Z",
  "roles": ["ROLE_USER"]
}
```

Behavior and reasons

name is nonblank and at most 100 characters; email is nonblank and syntactically valid; phone permits an optional plus and 7 to 15 digits; password is 8 to 100 characters. Email is trimmed and lowercased. Both email and phone must be unique. BCrypt hashes the password; ROLE_USER and default preferences are created. A six digit verification OTP is persisted and user events are published. No password or OTP is returned.

Other responses and failure conditions

400 duplicate email or phone or validation error; 409 database uniqueness conflict. No admin role is accepted from the request.

Source: UserController.register and UserService.register

API 2 Sign in

POST /api/auth/login

Access: Public

Additional mappings: POST /auth/login

Request body

```
{"email": "asha@example.com", "password": "ExamplePass123!"}
```

Success response example HTTP 200

```
{
  "accessToken": "<ACCESS_TOKEN>",
  "refreshToken": "<REFRESH_TOKEN>",
  "user": {
    "id": 1,
    "name": "Asha Sharma",
    "email": "asha@example.com",
    "phone": "+919876543210",
    "gender": null,
    "dob": null,
    "profileImage": null,
    "memberSince": "2026-09-15T10:00:00Z",
    "roles": ["ROLE_USER"]
  }
}
```

Behavior and reasons

UserService checks lock expiry and active status before BCrypt comparison. Failed attempts and login history are committed even when ApiException is thrown. The fifth wrong password locks the account for 30 minutes. Success resets failures, records device from User-Agent and remote IP, and returns a 15 minute default JWT plus a 7 day opaque refresh token.

Other responses and failure conditions

400 invalid credentials, missing user, locked or inactive account; field validation errors also reject malformed requests.

Source: AuthController.login and UserService.login

API 3 Rotate a refresh token

POST /api/auth/refresh

Access: Public endpoint requiring the refresh token in its body

Additional mappings: POST /auth/refresh

Request body

```
{"refreshToken": "<REFRESH_TOKEN>"}
```

Success response example HTTP 200

```
{
  "accessToken": "<ACCESS_TOKEN>",
  "refreshToken": "<REFRESH_TOKEN>",
  "user": {
    "id": 1,
    "name": "Asha Sharma",
    "email": "asha@example.com",
    "phone": "+919876543210",
    "gender": null,
    "dob": null,
    "profileImage": null,
    "memberSince": "2026-09-15T10:00:00Z",
    "roles": ["ROLE_USER"]
  }
}
```

Behavior and reasons

Reject missing, revoked, expired tokens and inactive or non-ACTIVE users. Revoke the old refresh row and create a new token with the same device. The access JWT is newly signed. The code does not lock this row or implement token family reuse detection, so concurrent refreshes are a known hardening gap.

Other responses and failure conditions

400 invalid, expired or revoked refresh token; 400 inactive account.

Source: AuthController.refresh and UserService.refresh

API 4 Request password reset

```
POST /api/auth/forgot-password
```

Access: Public

Additional mappings: POST /auth/forgot-password; POST /forgot-password

Request body

```
{"email": "asha@example.com"}
```

Success response example HTTP 202

Empty response body.

Behavior and reasons

Persist a six digit secure random OTP with a ten minute expiry and publish PasswordResetOtpRequested. The OTP is not in the HTTP response or event payload. Email delivery is not implemented. The method looks up the user and therefore does reveal a missing email through a 400 error.

Other responses and failure conditions

400 unknown or malformed email.

Source: AuthController.forgot and UserService.forgot

API 5 Reset a password using OTP

```
POST /api/auth/reset-password
```

Access: Public

Additional mappings: POST /auth/reset-password

Request body

```
{"email": "asha@example.com", "otp": "123456", "newPassword": "AnotherPass123!"}
```

Success response example HTTP 204

Empty response body.

Behavior and reasons

Find the latest unused token matching user and supplied OTP. Reject expired tokens, mark the selected token used, hash the new password, record passwordChangedOn and revoke all refresh tokens. A stored OTP is

required; the sample is a placeholder, not a working bypass. Existing access JWTs are not revoked by this operation.

Other responses and failure conditions

400 invalid OTP, expired OTP, unknown email or password validation failure.

Source: AuthController.reset and UserService.reset

API 6 Verify an email address

```
POST /api/auth/verify-email
```

Access: Public

Additional mappings: POST /auth/verify-email

Request body

```
{"email": "asha@example.com", "otp": "123456"}
```

Success response example HTTP 204

Empty response body.

Behavior and reasons

Locate the latest matching unused verification OTP, check its ten minute expiry, mark it used and set emailVerified=true. The current login method does not require this flag; verification is stored account metadata, not a login gate.

Other responses and failure conditions

400 invalid or expired OTP or missing user.

Source: AuthController.verify and UserService.verify

API 7 Sign out the supplied session

```
POST /api/auth/logout
```

Access: Bearer access token

```
X-Refresh-Token: <REFRESH_TOKEN>
```

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

X-Refresh-Token identifies the refresh row to revoke. The token must belong to the authenticated email; missing or foreign rows have no effect and still complete. Clearing the browser storage is separate. A still-valid access token remains cryptographically usable until expiry.

Other responses and failure conditions

401 missing authentication; missing required X-Refresh-Token is rejected by request binding.

Source: SessionController.logout and UserService.logout

API 8 Sign out all refresh sessions

```
POST /api/auth/logout-all
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

Revoke every refresh token for the authenticated user. This is not an access-token denylist and does not immediately log out other services holding unexpired JWTs.

Other responses and failure conditions

401 unauthenticated; 400 user not found.

Source: SessionController.logoutAll and UserService.logoutAll

API 9 Read your profile

```
GET /api/users/me
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "id": 1,
  "name": "Asha Sharma",
  "email": "asha@example.com",
  "phone": "+919876543210",
  "gender": null,
  "dob": null,
  "profileImage": null,
  "memberSince": "2026-09-15T10:00:00Z",
  "roles": ["ROLE_USER"]
}
```

Behavior and reasons

Resolve identity from the JWT email subject, not a userId query parameter. UserResponse intentionally omits the password and internal account security fields.

Other responses and failure conditions

401 unauthenticated; 400 user not found.

Source: UserController.me and UserService.me

API 10 Read a profile by identifier

```
GET /api/users/1
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "id": 1,
  "name": "Asha Sharma",
  "email": "asha@example.com",
  "phone": "+919876543210",
  "gender": null,
  "dob": null,
  "profileImage": null,
  "memberSince": "2026-09-15T10:00:00Z",
  "roles": ["ROLE_USER"]
}
```

Behavior and reasons

Path template is /api/users/{id}. IMPORTANT CURRENT BEHAVIOR: any authenticated user can call this endpoint for another ID. Neither ownership nor ROLE_ADMIN is checked. The response includes email and phone; restrict or remove this endpoint before public production use.

Other responses and failure conditions

401 unauthenticated; 400 user not found.

Source: UserController.get and UserService.get

API 11 Update your profile

```
PUT /api/users/me
```

Access: Bearer access token

Additional mappings: PUT /api/users/profile

Request body

```
{"name": "Asha S", "phone": "+919876543210", "dob": "1998-05-17", "gender": "Female"}
```

Success response example HTTP 200

```
{
  "id": 1,
  "name": "Asha S",
  "email": "asha@example.com",
  "phone": "+919876543210",
  "gender": "Female",
}
```

```
{
  "dob": "1998-05-17",
  "profileImage": null,
  "memberSince": "2026-09-15T10:00:00Z",
  "roles": ["ROLE_USER"]
}
```

Behavior and reasons

name and phone are required. A changed phone must be unique; name is at most 100 characters and phone follows the registration digit pattern. dob is an ISO date and gender has no enumerated validation. Null optional values overwrite prior values. Email and roles cannot be updated here.

Other responses and failure conditions

400 invalid phone, duplicate phone or missing user.

Source: UserController.update and UserService.update

API 12 Change your password

```
PUT /api/users/change-password
```

Access: Bearer access token

Request body

```
{"oldPassword": "ExamplePass123!", "newPassword": "AnotherPass123!"}
```

Success response example HTTP 204

Empty response body.

Behavior and reasons

Require the old password to match BCrypt. The new password must contain 8 to 100 characters; update the hash and passwordChangedOn. Unlike the reset flow, this method does not revoke refresh tokens. Neither password-change time nor active status is checked by downstream JWT filters.

Other responses and failure conditions

400 incorrect old password or validation failure; 401 unauthenticated.

Source: UserController.password and UserService.changePassword

API 13 Delete your account

```
DELETE /api/users/me
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

Soft delete: active=false and status=DELETED, revoke refresh tokens and publish AccountDeletedEvent. User rows and historical orders are retained. Existing signed access tokens are not checked against this flag by every service.

Other responses and failure conditions

401 unauthenticated; 400 user not found.

Source: UserController.delete and UserService.delete

API 14 Delete your account by identifier

```
DELETE /api/users/1
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

Path template is /api/users/{id}. Compare the requested ID to the authenticated profile ID before soft deletion. Even an admin token cannot use this endpoint to delete somebody else.

Other responses and failure conditions

400 You can only delete your own account; 401 unauthenticated.

Source: UserController.deleteById

API 15 List your addresses

```
GET /api/addresses
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
[
  {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  }
]
```

Source snapshot 15 September 2026

```
}
]
```

Behavior and reasons

Resolve user by JWT email and return only their addresses, default first and ID descending. The entity serializes inherited audit fields; @JsonIgnore prevents serialization of its User relationship.

Other responses and failure conditions

401 unauthenticated; 400 user not found.

Source: AddressController.list and AccountSettingsService.addresses

API 16 Add an address

POST /api/addresses

Access: Bearer access token

Request body

```
{
  "line1": "10 Example Road",
  "line2": "Apartment 2",
  "city": "Pune",
  "state": "Maharashtra",
  "postalCode": "411001",
  "country": "India"
}
```

Success response example HTTP 201

```
{
  "createdDate": "2026-09-15T10:00:00Z",
  "updatedAt": "2026-09-15T10:00:00Z",
  "createdBy": null,
  "updatedAt": null,
  "id": 1,
  "line1": "10 Example Road",
  "line2": "Apartment 2",
  "city": "Pune",
  "state": "Maharashtra",
  "postalCode": "411001",
  "country": "India",
  "defaultAddress": true
}
```

Behavior and reasons

line1, city, state, postalCode and country are nonblank; line2 is optional. The first stored address is marked default. No request field can select an owner. An AddressAddedEvent is published. No country-specific postal validation is implemented.

Other responses and failure conditions

400 required address fields missing; 401 unauthenticated.

Source: AddressController.add and AccountSettingsService.add

API 17 Update an address

```
PUT /api/addresses/1
```

Access: Bearer access token

Request body

```
{
  "line1": "20 Example Road",
  "line2": "Apartment 2",
  "city": "Pune",
  "state": "Maharashtra",
  "postalCode": "411001",
  "country": "India"
}
```

Success response example HTTP 200

```
{
  "createdDate": "2026-09-15T10:00:00Z",
  "updatedAt": "2026-09-15T10:00:00Z",
  "createdBy": null,
  "updatedBy": null,
  "id": 1,
  "line1": "20 Example Road",
  "line2": "Apartment 2",
  "city": "Pune",
  "state": "Maharashtra",
  "postalCode": "411001",
  "country": "India",
  "defaultAddress": true
}
```

Behavior and reasons

Path template is `/api/addresses/{id}`. The service finds the address, compares its owner to the authenticated user, then replaces address fields. The default flag is not changed through this request.

Other responses and failure conditions

400 Address not found for missing or foreign address; validation errors; 401 unauthenticated.

Source: `AddressController.update` and `AccountSettingsService.update`

API 18 Remove an address

```
DELETE /api/addresses/1
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

Delete only an owned address. Previously created orders keep their copied addressJson. Deleting the default address does not automatically choose another default.

Other responses and failure conditions

400 Address not found for missing or foreign address; 401 unauthenticated.

Source: AddressController.delete and AccountSettingsService.delete

API 19 Set a default address

PATCH /api/addresses/1/default

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "createdAt": "2026-09-15T10:00:00Z",
  "updatedAt": "2026-09-15T10:00:00Z",
  "createdBy": null,
  "updatedBy": null,
  "id": 1,
  "line1": "10 Example Road",
  "line2": "Apartment 2",
  "city": "Pune",
  "state": "Maharashtra",
  "postalCode": "411001",
  "country": "India",
  "defaultAddress": true
}
```

Behavior and reasons

Verify ownership. If already default, return immediately. Otherwise clear that user's existing default flags and set this address default. Concurrent requests are not protected by a database unique-default constraint or explicit user lock.

Other responses and failure conditions

400 Address not found; 401 unauthenticated.

Source: AddressController.defaultAddress and AccountSettingsService.makeDefault

API 20 Read preferences

GET /api/users/preferences

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "id": 1,
  "language": "en",
  "currency": "INR",
  "darkMode": false,
  "emailNotifications": true,
  "smsNotifications": true,
  "whatsappNotifications": false,
  "pushNotifications": true,
  "communicationPreference": "EMAIL"
}
```

Behavior and reasons

Return the authenticated user's preference row. If absent, the GET creates a default row; this is an intentional implementation detail to mention when discussing HTTP safe methods. Currency preference does not change order currency, which remains INR.

Other responses and failure conditions

401 unauthenticated; 400 user not found.

Source: PreferencesController.get and AccountSettingsService.preferences

API 21 Update preferences

```
PUT /api/users/preferences
```

Access: Bearer access token

Request body

```
{
  "language": "en",
  "currency": "INR",
  "darkMode": true,
  "emailNotifications": true,
  "smsNotifications": false,
  "whatsappNotifications": false,
  "pushNotifications": true,
  "communicationPreference": "EMAIL"
}
```

Success response example HTTP 200

```
{
  "id": 1,
  "language": "en",
  "currency": "INR",
  "darkMode": true,
  "emailNotifications": true,
  "smsNotifications": false,
  "whatsappNotifications": false,
  "pushNotifications": true,
  "communicationPreference": "EMAIL"
}
```

Behavior and reasons

Only non-null request fields are applied, so this PUT behaves as a partial update. There is no @Valid or enum validation on this controller request. Flags store preferences; they do not configure an SMS/email provider or control the current notification consumer.

Other responses and failure conditions

401 unauthenticated; malformed JSON rejected by binding.

Source: PreferencesController.update and AccountSettingsService.updatePreferences

API 22 Direct service greeting

```
GET /hello
```

Access: Public on user-service port 8082

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
User Service is running...
```

Behavior and reasons

A plain text greeting, not a dependency readiness check. No explicit gateway route exists for /hello. Do not infer MySQL, Kafka or payment health from this response.

Other responses and failure conditions

No custom business failure path.

Source: HealthController.hello

Product service

API 23 Search the public catalog

GET

```
/api/v1/products?query=phone&category=Electronics&minPrice=100&maxPrice=2000&inStock=true&page=0&size=20&sort=price,asc
```

Access: Public

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "content": [
    {
      "id": 1,
      "sku": "PHONE-001",
      "name": "Everyday Phone",
      "description": "A sample catalog product",
      "price": 1000.0,
      "stockQuantity": 10,
      "category": "Electronics",
      "active": true,
      "imageUrl": "https://example.com/phone.jpg",
      "createdAt": "2026-09-15T10:00:00Z",
      "updatedAt": "2026-09-15T10:00:00Z"
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": {"sorted": true, "unsorted": false, "empty": false},
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "last": true,
  "totalPages": 1,
  "totalElements": 1,
  "size": 20,
  "number": 0,
  "sort": {"sorted": true, "unsorted": false, "empty": false},
  "first": true,
  "numberOfElements": 1,
  "empty": false
}
```

Behavior and reasons

All filters are optional and combined with AND. Name search and category match are case-insensitive; search is a contains LIKE query and literal %, _ and backslash are escaped. Category is an exact match. Price bounds are inclusive and nonnegative. inStock=true means quantity > 0; false means quantity = 0. Hidden products are always excluded, including for admin callers of this public listing. Default page size is 20 and sort createdAt ascending. Size is capped at 100. Sort permits id, name, price, createdAt and stockQuantity; id is appended as a stable tie-breaker.

Other responses and failure conditions

400 negative/inverted price range or unsupported sort; invalid bearer tokens are rejected even on public routes.

Source: ProductController.getProducts and ProductSearch.search

API 24 Read a product

GET /api/v1/products/1

Access: Public for active products; ROLE_ADMIN can view inactive products

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "id": 1,
  "sku": "PHONE-001",
  "name": "Everyday Phone",
  "description": "A sample catalog product",
  "price": 1000.0,
  "stockQuantity": 10,
  "category": "Electronics",
  "active": true,
  "imageUrl": "https://example.com/phone.jpg",
  "createdAt": "2026-09-15T10:00:00Z",
  "updatedAt": "2026-09-15T10:00:00Z"
}
```

Behavior and reasons

Fetch current JPA version from MySQL, then optionally load the versioned Redis entry. ProductController hides inactive products from non-admin callers with 404. The service method itself can still return inactive products for internal Java use.

Other responses and failure conditions

404 missing or hidden product; 401 invalid supplied bearer token.

Source: ProductController.getProduct and ProductServiceImpl.getProduct

API 25 Create a product

POST /api/v1/products

Access: Bearer token with ROLE_ADMIN

Request body

```
{
  "sku": "PHONE-001",
  "name": "Everyday Phone",
  "description": "A sample catalog product",
  "price": 1000.0,
  "stockQuantity": 10,
  "category": "Electronics",
  "active": true,
  "imageUrl": "https://example.com/phone.jpg"
}
```

Success response example HTTP 201

```
{
  "id": 1,
  "sku": "PHONE-001",
  "name": "Everyday Phone",
  "description": "A sample catalog product",
  "price": 1000.0,
  "stockQuantity": 10,
  "category": "Electronics",
  "active": true,
  "imageUrl": "https://example.com/phone.jpg",
  "createdAt": "2026-09-15T10:00:00Z",
  "updatedAt": "2026-09-15T10:00:00Z"
}
```

Behavior and reasons

SKU is nonblank, at most 64 characters, letters/digits/underscore/hyphen, normalized to uppercase and unique. Name is nonblank and <=150; description <=2000; price is strictly positive with at most 17 integer and 2 fractional digits. Stock is required, 0 to 1000000. Category <=100; active defaults true when null. imageUrl is optional, <=2048 and empty or HTTPS. The response has a Location header pointing to the created product.

Other responses and failure conditions

401 missing or invalid token; 403 non-admin; 400 validation; 409 duplicate SKU.

Source: ProductController.createProduct and ProductServiceImpl.createProduct

API 26 Replace product details

```
PUT /api/v1/products/1
```

Access: Bearer token with ROLE_ADMIN

Request body

```
{
  "sku": "PHONE-001",
  "name": "Everyday Phone",
  "description": "A sample catalog product",
  "price": 1000.0,
  "stockQuantity": 8,
  "category": "Electronics",
  "active": true,
  "imageUrl": "https://example.com/phone.jpg"
}
```

Success response example HTTP 200

```
{
  "id": 1,
  "sku": "PHONE-001",
  "name": "Everyday Phone",
  "description": "A sample catalog product",
  "price": 1000.0,
  "stockQuantity": 8,
  "category": "Electronics",
  "active": true,
  "imageUrl": "https://example.com/phone.jpg",
  "createdAt": "2026-09-15T10:00:00Z",
  "updatedAt": "2026-09-15T10:00:00Z"
}
```

```
"updatedAt": "2026-09-15T10:00:00Z"
}
```

Behavior and reasons

Same validation as creation. Exclude the current ID when checking duplicate SKU. The stock value sets currently available stock, not original purchased quantity. @Version detects overlapping writes at transaction time; the API does not require an ETag from a long-open admin form.

Other responses and failure conditions

404 missing; 409 duplicate SKU or optimistic conflict; 400 validation; 401/403 access control.

Source: ProductController.updateProduct and ProductServiceImpl.updateProduct

API 27 Delete an unreserved product

```
DELETE /api/v1/products/1
```

Access: Bearer token with ROLE_ADMIN

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

Lock the product row. Reject deletion when a RESERVED or COMMITTED reservation contains it, so existing checkout and return restocking retain their product. Deactivate through PUT instead. A product whose reservations were released or returned can be deleted.

Other responses and failure conditions

409 reserved or fulfilled stock; 404 missing; 401/403 access control.

Source: ProductServiceImpl.deleteProduct and ReservationRepository.hasReservation

API 28 List the admin catalog

```
GET /api/v1/products/admin/catalog?page=0&size=20
```

Access: Bearer token with ROLE_ADMIN

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "content": [
    {
      "id": 1,
      "sku": "PHONE-001",
      "name": "Everyday Phone",
      "description": "A sample catalog product",
      "price": 1000.0,
      "stockQuantity": 10,

```

```

    "category": "Electronics",
    "active": true,
    "imageUrl": "https://example.com/phone.jpg",
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z"
  }
],
"pageable": {
  "pageNumber": 0,
  "pageSize": 20,
  "sort": {"sorted": true, "unsorted": false, "empty": false},
  "offset": 0,
  "paged": true,
  "unpaged": false
},
"last": true,
"totalPages": 1,
"totalElements": 1,
"size": 20,
"number": 0,
"sort": {"sorted": true, "unsorted": false, "empty": false},
"first": true,
"numberOfElements": 1,
"empty": false
}

```

Behavior and reasons

List active and inactive products. This endpoint accepts Pageable parameters; it does not expose the public search filters. It uses the same size cap, sort allowlist and id tie-breaker.

Other responses and failure conditions

401/403; 400 invalid sort.

Source: ProductController.adminCatalog and ProductSecurityConfig

Cart service

API 29 Read your cart

GET /api/cart

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "userId": 1,
  "items": [
    {
      "productId": 1,
      "name": "Everyday Phone",
      "quantity": 2,
      "unitPrice": 1000.0,
      "subtotal": 2000.0,
      "available": true
    }
  ],
  "totalQuantity": 2,
  "totalPrice": 2000.0,
  "version": 3
}
```

Behavior and reasons

Load cart by authenticated numeric userId. Read current product details for each line; calculate subtotal and total with BigDecimal. Missing or hidden products remain removable as unavailable lines with null prices, excluded from totals. An absent cart returns an empty response with version null. version distinguishes later purchases of identical contents.

Other responses and failure conditions

401; 503 product dependency failure; missing product lines do not fail the complete response.

Source: CartController.get and CartService.get

API 30 Add quantity to your cart

POST /api/cart/items

Access: Bearer access token

Request body

```
{"productId": 1, "quantity": 2}
```

Success response example HTTP 200

```
{
  "userId": 1,
  "items": [
    {
      "productId": 1,
```

```

    "name": "Everyday Phone",
    "quantity": 2,
    "unitPrice": 1000.0,
    "subtotal": 2000.0,
    "available": true
  }
],
"totalQuantity": 2,
"totalPrice": 2000.0,
"version": 3
}

```

Behavior and reasons

Both values are required and positive. Quantity is added to the existing amount. Validate the combined quantity against current active product stock and integer range. Persist/flush the cart and return the complete calculated cart. This POST is not generally idempotent; do not automatically replay after an unknown outcome.

Other responses and failure conditions

400 invalid quantity; 404 product missing; 409 insufficient stock or concurrent cart conflict; 503 product unavailable.

Source: CartController.add and CartService.add

API 31 Set a cart line quantity

PUT /api/cart/items/1

Access: Bearer access token

Request body

```

{"quantity": 2}

```

Success response example HTTP 200

```

{
  "userId": 1,
  "items": [
    {
      "productId": 1,
      "name": "Everyday Phone",
      "quantity": 2,
      "unitPrice": 1000.0,
      "subtotal": 2000.0,
      "available": true
    }
  ],
  "totalQuantity": 2,
  "totalPrice": 2000.0,
  "version": 3
}

```

Behavior and reasons

Replace rather than increment quantity. The item must already exist in this user's cart. Required positive quantity is checked against current stock. @Version on Cart and a unique user/product element collection entry protect persistence; concurrency failures are surfaced as 409.

Other responses and failure conditions

404 cart item missing; 409 insufficient stock or concurrency; 400 validation; 503 dependency failure.

Source: CartController.update and CartService.update

API 32 Remove one cart line

```
DELETE /api/cart/items/1
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

Remove only from the authenticated cart; missing cart or missing line is a no-op. The operation does not reserve or release inventory because cart additions never reserved stock.

Other responses and failure conditions

401; concurrent modification can produce 409.

Source: CartController.remove and CartService.remove

API 33 Empty your cart

```
DELETE /api/cart
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

Clear the authenticated cart item collection. Missing cart is a no-op. The cart entity remains and its version changes when an existing item collection changes.

Other responses and failure conditions

401; concurrent modification can produce 409.

Source: CartController.clear and CartService.clear

Order service

API 34 Create or resume checkout

POST /api/orders

Access: Bearer access token

Idempotency-Key: checkout-demo-0001

Request body

```
{"addressId": 1, "paymentMethod": "CASH_ON_DELIVERY", "couponCode": "SAVE10"}
```

Success response example HTTP 200

```
{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "CONFIRMED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "UNPAID",
  "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
  "checkout": null,
  "createdAt": "2026-09-15T10:00:00Z",
  "expiresAt": null,
  "refund": {"status": "NONE", "razorpayRefundId": null, "failure": null},
  "carrier": null,
  "trackingNumber": null,
  "shippedAt": null,
  "deliveredAt": null,
  "subtotal": 2000.0,
  "discount": 200.0,
  "couponCode": "SAVE10",
  "returnRequest": {
    "status": "NONE",
    "reason": null,
    "decisionNote": null,
    "requestedAt": null,
    "receivedAt": null,
    "restock": null,
    "manualRefundReference": null
  }
}
```

Behavior and reasons

addressId must be positive; paymentMethod is CASH_ON_DELIVERY or RAZORPAY. Optional couponCode is 3 to 40 ASCII letters/digits/underscore/hyphen. The required key is 8 to 128 characters from that same character set. Customer identity, cart contents, product prices and shipping address ownership are resolved server-side. The key is hashed and unique per user. The same key must keep address, payment method and normalized coupon unchanged. A second key cannot buy the same cart version. COD confirms after reservation; online setup may remain pending. Response includes Location: /api/orders/{id}.

Other responses and failure conditions

200 known usable state; 202 CREATING, RESERVED or CANCELLING; 409 FAILED, changed key details, duplicate cart version or invalid coupon; 404 foreign address; 503 dependencies or missing test payment configuration.

Source: OrderController.create and OrderService.create

API 35 List your orders

```
GET /api/orders?page=0&size=20
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "content": [
    {
      "id": "11111111-1111-4111-8111-111111111111",
      "status": "CONFIRMED",
      "paymentMethod": "CASH_ON_DELIVERY",
      "paymentStatus": "UNPAID",
      "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
      "totalPrice": 1800.0,
      "currency": "INR",
      "shippingAddress": {
        "createdDate": "2026-09-15T10:00:00Z",
        "updatedAt": "2026-09-15T10:00:00Z",
        "createdBy": null,
        "updatedAt": null,
        "id": 1,
        "line1": "10 Example Road",
        "line2": "Apartment 2",
        "city": "Pune",
        "state": "Maharashtra",
        "postalCode": "411001",
        "country": "India",
        "defaultAddress": true
      },
      "cartCleared": true,
      "failureReason": null,
      "checkout": null,
      "createdAt": "2026-09-15T10:00:00Z",
      "expiresAt": null,
      "refund": {"status": "NONE", "razorpayRefundId": null, "failure": null},
      "carrier": null,
      "trackingNumber": null,
      "shippedAt": null,
      "deliveredAt": null,
    }
  ]
}
```

```

    "subtotal": 2000.0,
    "discount": 200.0,
    "couponCode": "SAVE10",
    "returnRequest": {
      "status": "NONE",
      "reason": null,
      "decisionNote": null,
      "requestedAt": null,
      "receivedAt": null,
      "restock": null,
      "manualRefundReference": null
    }
  },
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": {"sorted": false, "unsorted": true, "empty": true},
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "last": true,
  "totalPages": 1,
  "totalElements": 1,
  "size": 20,
  "number": 0,
  "sort": {"sorted": false, "unsorted": true, "empty": true},
  "first": true,
  "numberOfElements": 1,
  "empty": false
}

```

Behavior and reasons

Only the authenticated user's orders, newest first. page ≥ 0 and size 1 to 100, default 20. GET reads persisted state; it does not itself call advance or force payment reconciliation.

Other responses and failure conditions

401; invalid page/size rejected.

Source: OrderController.list and OrderService.list

API 36 Read an owned order

GET /api/orders/11111111-1111-4111-8111-111111111111

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```

{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "CONFIRMED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "UNPAID",
  "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
  "totalPrice": 1800.0,

```

```

"currency": "INR",
"shippingAddress": {
  "createdAt": "2026-09-15T10:00:00Z",
  "updatedAt": "2026-09-15T10:00:00Z",
  "createdBy": null,
  "updatedBy": null,
  "id": 1,
  "line1": "10 Example Road",
  "line2": "Apartment 2",
  "city": "Pune",
  "state": "Maharashtra",
  "postalCode": "411001",
  "country": "India",
  "defaultAddress": true
},
"cartCleared": true,
"failureReason": null,
"checkout": null,
"createdAt": "2026-09-15T10:00:00Z",
"expiresAt": null,
"refund": {"status": "NONE", "razorpayRefundId": null, "failure": null},
"carrier": null,
"trackingNumber": null,
"shippedAt": null,
"deliveredAt": null,
"subtotal": 2000.0,
"discount": 200.0,
"couponCode": "SAVE10",
"returnRequest": {
  "status": "NONE",
  "reason": null,
  "decisionNote": null,
  "requestedAt": null,
  "receivedAt": null,
  "restock": null,
  "manualRefundReference": null
}
}

```

Behavior and reasons

UUID path parameter. A missing or foreign order returns the same 404. Snapshot fields preserve the prices, discount and address captured at purchase; they are not replaced when catalog/profile data changes.

Other responses and failure conditions

404 missing or foreign order; 400 malformed UUID; 401.

Source: OrderController.get and OrderService.owned

API 37 Cancel an eligible order

```
POST /api/orders/11111111-1111-4111-8111-111111111111/cancel
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "CANCELLED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "UNPAID",
  "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdDate": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedAt": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
  "checkout": null,
  "createdAt": "2026-09-15T10:00:00Z",
  "expiresAt": null,
  "refund": {"status": "NONE", "razorpayRefundId": null, "failure": null},
  "carrier": null,
  "trackingNumber": null,
  "shippedAt": null,
  "deliveredAt": null,
  "subtotal": 2000.0,
  "discount": 200.0,
  "couponCode": "SAVE10",
  "returnRequest": {
    "status": "NONE",
    "reason": null,
    "decisionNote": null,
    "requestedAt": null,
    "receivedAt": null,
    "restock": null,
    "manualRefundReference": null
  }
}
```

Behavior and reasons

Allow CREATING, RESERVED, PENDING_PAYMENT, CONFIRMED or PROCESSING. Record CANCELLING first, then release reservation, finalize CANCELLED and release coupon use. Paid online orders request a refund. Closed/FAILED states return idempotently; shipped/delivered orders require the return flow instead.

Other responses and failure conditions

202 while cancelling; 409 invalid transition; 404 foreign or missing order; 401.

Source: OrderController.cancel and OrderLifecycle.cancel

API 38 Verify a Razorpay browser callback

POST /api/orders/11111111-1111-4111-8111-111111111111/payments/verify

Access: Bearer access token

Request body

```
{
  "razorpay_order_id": "order_demo1",
  "razorpay_payment_id": "pay_demo1",
  "razorpay_signature": "<64_HEX_CHARACTERS>"
}
```

Success response example HTTP 200

```
{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "CONFIRMED",
  "paymentMethod": "RAZORPAY",
  "paymentStatus": "PAID",
  "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
  "checkout": null,
  "createdAt": "2026-09-15T10:00:00Z",
  "expiresAt": "2026-09-15T10:15:00Z",
  "refund": {"status": "NONE", "razorpayRefundId": null, "failure": null},
  "carrier": null,
  "trackingNumber": null,
  "shippedAt": null,
  "deliveredAt": null,
  "subtotal": 2000.0,
  "discount": 200.0,
  "couponCode": "SAVE10",
  "returnRequest": {
    "status": "NONE",
    "reason": null,
    "decisionNote": null,
    "requestedAt": null,
    "receivedAt": null,
    "restock": null,
    "manualRefundReference": null
  }
}
```

Behavior and reasons

Provider order and payment IDs must match order_ and pay_ alphanumeric formats; signature must contain 64 hex characters. Use the stored provider order ID for HMAC, not an arbitrary browser ID. Fetch provider payment

and provider order, require captured/refunded payment with exact order, amount and INR, and a paid order. Late captures keep a closed order closed and request refund.

Other responses and failure conditions

400 bad signature or payment/order mismatch; 409 not captured or conflicting payment; 404 foreign order; 503 provider issue.

Source: OrderController.verify and OrderService.verify

API 39 Receive a signed payment webhook

POST /api/payments/razorpay/webhook

Access: Raw-body HMAC in X-Razorpay-Signature instead of customer JWT

X-Razorpay-Signature: <HMAC_SHA256_OF_EXACT_BODY>

Request body

```
{
  "event": "payment.captured",
  "payload": { "payment": { "entity": { "id": "pay_demo1", "order_id": "order_demo1" } } }
}
```

Success response example HTTP 204

Empty response body.

Behavior and reasons

Verify exact raw bytes before parsing JSON. Accept payment.captured, order.paid, refund.created, refund.processed and refund.failed. Other events are acknowledged without action. Unknown provider orders are ignored. Re-query provider state instead of trusting event amount or a browser success flag. A minimal consumed-fields sample is shown; real Razorpay envelopes have additional fields.

Other responses and failure conditions

400 invalid signature or JSON; 503 missing webhook secret or surfaced provider error. Payment advance catches recoverable failures, so a 204 can mean scheduled recovery will finish the work.

Source: PaymentWebhookController.webhook and OrderService.webhook

API 40 List orders as an administrator

GET /api/admin/orders?page=0&size=20

Access: Bearer token with ROLE_ADMIN

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "content": [
    {
      "id": "11111111-1111-4111-8111-111111111111",
      "status": "CONFIRMED",

```

```

    "paymentMethod": "CASH_ON_DELIVERY",
    "paymentStatus": "UNPAID",
    "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
    "totalPrice": 1800.0,
    "currency": "INR",
    "shippingAddress": {
      "createdAt": "2026-09-15T10:00:00Z",
      "updatedAt": "2026-09-15T10:00:00Z",
      "createdBy": null,
      "updatedBy": null,
      "id": 1,
      "line1": "10 Example Road",
      "line2": "Apartment 2",
      "city": "Pune",
      "state": "Maharashtra",
      "postalCode": "411001",
      "country": "India",
      "defaultAddress": true
    },
    "cartCleared": true,
    "failureReason": null,
    "checkout": null,
    "createdAt": "2026-09-15T10:00:00Z",
    "expiresAt": null,
    "refund": {"status": "NONE", "razorpayRefundId": null, "failure": null},
    "carrier": null,
    "trackingNumber": null,
    "shippedAt": null,
    "deliveredAt": null,
    "subtotal": 2000.0,
    "discount": 200.0,
    "couponCode": "SAVE10",
    "returnRequest": {
      "status": "NONE",
      "reason": null,
      "decisionNote": null,
      "requestedAt": null,
      "receivedAt": null,
      "restock": null,
      "manualRefundReference": null
    }
  }
],
"pageable": {
  "pageNumber": 0,
  "pageSize": 20,
  "sort": {"sorted": true, "unsorted": false, "empty": false},
  "offset": 0,
  "paged": true,
  "unpaged": false
},
"last": true,
"totalPages": 1,
"totalElements": 1,
"size": 20,
"number": 0,
"sort": {"sorted": true, "unsorted": false, "empty": false},
"first": true,
"numberOfElements": 1,
"empty": false
}

```

Behavior and reasons

Return all orders, newest first. page ≥ 0 , size 1 to 100. The current OrderView does not include userId, although its copied delivery address is visible to admins.

Other responses and failure conditions

401/403; invalid page or size.

Source: AdminOrderController.list and OrderService.adminList

API 41 Read any order as an administrator

```
GET /api/admin/orders/11111111-1111-4111-8111-111111111111
```

Access: Bearer token with ROLE_ADMIN

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "CONFIRMED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "UNPAID",
  "items": [{ "productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2 }],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
  "checkout": null,
  "createdAt": "2026-09-15T10:00:00Z",
  "expiresAt": null,
  "refund": { "status": "NONE", "razorpayRefundId": null, "failure": null },
  "carrier": null,
  "trackingNumber": null,
  "shippedAt": null,
  "deliveredAt": null,
  "subtotal": 2000.0,
  "discount": 200.0,
  "couponCode": "SAVE10",
  "returnRequest": {
    "status": "NONE",
    "reason": null,
    "decisionNote": null,
    "requestedAt": null,
    "receivedAt": null,
  }
}
```

```

    "restock": null,
    "manualRefundReference": null
  }
}

```

Behavior and reasons

UUID validation and ROLE_ADMIN at the controller class. This route intentionally does not apply customer ownership filtering.

Other responses and failure conditions

401/403; 404 missing order; 400 invalid UUID.

Source: AdminOrderController.get and OrderService.adminGet

API 42 Advance order fulfillment

PATCH /api/admin/orders/11111111-1111-4111-8111-111111111111/status

Access: Bearer token with ROLE_ADMIN

Request body

```

{"status": "SHIPPED", "carrier": "Example Courier", "trackingNumber": "TRACK001"}

```

Success response example HTTP 200

```

{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "SHIPPED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "UNPAID",
  "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
  "checkout": null,
  "createdAt": "2026-09-15T10:00:00Z",
  "expiresAt": null,
  "refund": {"status": "NONE", "razorpayRefundId": null, "failure": null},
  "carrier": "Example Courier",
  "trackingNumber": "TRACK001",
  "shippedAt": "2026-09-15T10:00:00Z",
  "deliveredAt": null,
  "subtotal": 2000.0,
  "discount": 200.0,
  "couponCode": "SAVE10",

```

```

"returnRequest": {
  "status": "NONE",
  "reason": null,
  "decisionNote": null,
  "requestedAt": null,
  "receivedAt": null,
  "restock": null,
  "manualRefundReference": null
}
}

```

Behavior and reasons

Allowed target enum: PROCESSING, SHIPPED, DELIVERED. Preconditions are CONFIRMED, PROCESSING and SHIPPED respectively. Online orders must be PAID. SHIPPED requires nonblank carrier <=100 and trackingNumber <=150. DELIVERED COD requires cashCollected=true. Repeating a shipping transition with different carrier/tracking is 409. Shipping/delivery triggers inventory commit; the state update can already be persisted if that remote commit fails, so retry/refresh is necessary.

Other responses and failure conditions

409 invalid transition, unpaid online order or changed shipping details; 400 missing shipment fields or COD confirmation; 401/403; 404 missing.

Source: AdminOrderController.fulfill and OrderLifecycle.fulfill

API 43 Retry a failed online refund

POST /api/admin/orders/11111111-1111-4111-8111-111111111111/refund/retry

Access: Bearer token with ROLE_ADMIN

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```

{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "CANCELLED",
  "paymentMethod": "RAZORPAY",
  "paymentStatus": "REFUND_PENDING",
  "items": [{ "productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2 }],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdDate": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
}

```

```

"checkout": null,
"createdAt": "2026-09-15T10:00:00Z",
"expiresAt": null,
"refund": {"status": "PENDING", "razorpayRefundId": "rfnd_demo1", "failure": null},
"carrier": null,
"trackingNumber": null,
"shippedAt": null,
"deliveredAt": null,
"subtotal": 2000.0,
"discount": 200.0,
"couponCode": "SAVE10",
"returnRequest": {
  "status": "NONE",
  "reason": null,
  "decisionNote": null,
  "requestedAt": null,
  "receivedAt": null,
  "restock": null,
  "manualRefundReference": null
}
}

```

Behavior and reasons

Require a closed order with refundStatus FAILED. If a provider refund ID exists, first read it. Create a new attempt only after the previous provider refund is confirmed failed. Uncertain calls reuse the existing attempt UUID and identical request body. A retry response can remain PENDING.

Other responses and failure conditions

409 nonfailed/nonclosed refund; 503 provider failure; 401/403; 404 missing order.

Source: AdminOrderController.retryRefund and OrderLifecycle.retryRefund

Reviews and wishlist

API 44 Read public product reviews

```
GET /api/reviews/products/1?page=0&size=10
```

Access: Public GET

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "averageRating": 5.0,
  "reviewCount": 1,
  "reviews": {
    "content": [
      {
        "id": 1,
        "rating": 5,
        "comment": "Useful for everyday work.",
        "createdAt": "2026-09-15T10:00:00Z",
        "verifiedPurchase": true
      }
    ],
    "pageable": {
      "pageNumber": 0,
      "pageSize": 10,
      "sort": {"sorted": false, "unsorted": true, "empty": true},
      "offset": 0,
      "paged": true,
      "unpaged": false
    },
    "last": true,
    "totalPages": 1,
    "totalElements": 1,
    "size": 10,
    "number": 0,
    "sort": {"sorted": false, "unsorted": true, "empty": true},
    "first": true,
    "numberOfElements": 1,
    "empty": false
  }
}
```

Behavior and reasons

Positive product ID, page ≥ 0 and size 1 to 100; default size 10. Return aggregate average and count plus newest-first reviews. Customer and order IDs are omitted from ReviewView. The service does not validate that the product still exists; an unknown product ID can return an empty collection with average zero.

Other responses and failure conditions

400 invalid parameters; supplied invalid JWT can produce 401.

Source: ShoppingController.reviews and ShoppingService.reviews

API 45 Review a delivered purchase

```
POST /api/orders/11111111-1111-4111-8111-111111111111/reviews
```

Source snapshot 15 September 2026

Access: Bearer access token

Request body

```
{"productId": 1, "rating": 5, "comment": "Useful for everyday work."}
```

Success response example HTTP 201

```
{
  "id": 1,
  "rating": 5,
  "comment": "Useful for everyday work.",
  "createdAt": "2026-09-15T10:00:00Z",
  "verifiedPurchase": true
}
```

Behavior and reasons

Positive product ID; rating 1 to 5; nonblank comment <=2000. Lock and verify order ownership. Require deliveredAt and the product in itemsJson. Enforce one review per user/product through an existence check plus database uniqueness. A returned order with a delivery timestamp remains reviewable. No edit/delete/moderation endpoint is implemented.

Other responses and failure conditions

404 foreign order; 409 not delivered or duplicate review; 400 nonpurchased product or invalid rating/comment.

Source: ShoppingController.review and ShoppingService.review

API 46 Read saved products

```
GET /api/wishlist?page=0&size=20
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "content": [{"id": 1, "userId": 1, "productId": 1, "createdAt": "2026-09-15T10:00:00Z"}],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": {"sorted": false, "unsorted": true, "empty": true},
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "last": true,
  "totalPages": 1,
  "totalElements": 1,
  "size": 20,
  "number": 0,
  "sort": {"sorted": false, "unsorted": true, "empty": true},
  "first": true,
  "numberOfElements": 1,
  "empty": false
}
```

Behavior and reasons

Only own wishlist entries, newest first. Entries contain product IDs, not catalog snapshots. The browser fetches each product; missing/hidden saved products can still be removed. page ≥ 0 ; size 1 to 100.

Other responses and failure conditions

401; invalid pagination.

Source: ShoppingController.wishlist and ShoppingService.wishlist

API 47 Save a product

```
PUT /api/wishlist/1
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{"id": 1, "userId": 1, "productId": 1, "createdAt": "2026-09-15T10:00:00Z"}
```

Behavior and reasons

Positive path ID. Return existing own entry before checking the product again, making sequential repeats safe. New entries require an active product via product-service. The unique user/product constraint prevents duplicates; simultaneous first inserts may return a 409 to one caller.

Other responses and failure conditions

404 missing/inactive product; 503 product service unavailable; 409 concurrent uniqueness conflict; 401.

Source: ShoppingController.save and ShoppingService.save

API 48 Remove a saved product

```
DELETE /api/wishlist/1
```

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

Delete by authenticated user and product ID; absent entries are harmless. The browser moves an item to the bag through separate cart read/add and wishlist delete calls.

Other responses and failure conditions

401; positive path validation.

Source: ShoppingController.remove and ShoppingService.remove

Coupons

API 49 Estimate a coupon discount

POST /api/coupons/quote

Access: Bearer access token

Request body

```
{"code": "SAVE10", "subtotal": 2000.0}
```

Success response example HTTP 200

```
{"code": "SAVE10", "subtotal": 2000.0, "discount": 200.0, "total": 1800.0}
```

Behavior and reasons

Code 3 to 40 allowed characters; subtotal ≥ 0.01 with at most 14 integer and 2 fractional digits. Validate active flag, expiry, usage and minimum spend. This caller-supplied amount is only an estimate and never authorizes the paid amount. No usage is consumed by a quote.

Other responses and failure conditions

409 unknown, inactive, expired, exhausted or minimum-spend failure; 400 validation; 401.

Source: CouponController.quote and CouponService.quote

API 50 Create a discount rule

POST /api/admin/coupons

Access: Bearer token with ROLE_ADMIN

Request body

```
{
  "code": "SAVE10",
  "type": "PERCENT",
  "value": 10.0,
  "minimumSpend": 500.0,
  "expiresAt": "2027-12-31T23:59:59Z",
  "usageLimit": 100
}
```

Success response example HTTP 200

```
{
  "code": "SAVE10",
  "type": "PERCENT",
  "value": 10.0,
  "minimumSpend": 500.0,
  "expiresAt": "2027-12-31T23:59:59Z",
  "usageLimit": 100,
  "usedCount": 0,
  "active": true,
  "version": 0
}
```

Behavior and reasons

type PERCENT or FIXED; positive value with ≤ 12 integer/2 fractional digits; nonnegative minimumSpend with same digit limits; expiry must be future; usageLimit 1 to 1000000. Percent cannot exceed 100. Normalize code uppercase and reject existing code. Current controller returns 200, not 201. Rule values cannot be edited through another endpoint.

Other responses and failure conditions

400 invalid rule; 409 duplicate code; 401/403.

Source: CouponController.create and CouponService.create

API 51 List discount rules

```
GET /api/admin/coupons?page=0&size=20
```

Access: Bearer token with ROLE_ADMIN

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "content": [
    {
      "code": "SAVE10",
      "type": "PERCENT",
      "value": 10.0,
      "minimumSpend": 500.0,
      "expiresAt": "2027-12-31T23:59:59Z",
      "usageLimit": 100,
      "usedCount": 1,
      "active": true,
      "version": 1
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": {"sorted": true, "unsorted": false, "empty": false},
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "last": true,
  "totalPages": 1,
  "totalElements": 1,
  "size": 20,
  "number": 0,
  "sort": {"sorted": true, "unsorted": false, "empty": false},
  "first": true,
  "numberOfElements": 1,
  "empty": false
}
```

Behavior and reasons

All rules, including inactive/expired ones, ordered by expiry descending. Includes usedCount and JPA version. page ≥ 0 ; size 1 to 100.

Other responses and failure conditions

401/403; invalid pagination.

Source: CouponController.list and CouponService.list

API 52 Activate or deactivate a rule

```
PATCH /api/admin/coupons/SAVE10
```

Access: Bearer token with ROLE_ADMIN

Request body

```
{"active": false}
```

Success response example HTTP 200

```
{
  "code": "SAVE10",
  "type": "PERCENT",
  "value": 10.0,
  "minimumSpend": 500.0,
  "expiresAt": "2027-12-31T23:59:59Z",
  "usageLimit": 100,
  "usedCount": 1,
  "active": false,
  "version": 2
}
```

Behavior and reasons

Require a non-null Boolean active. Lock the coupon row before changing it. Activating an already expired code does not extend its expiry; validation still rejects it. Existing order price snapshots are unchanged.

Other responses and failure conditions

409 unknown coupon; 400 null active; 401/403.

Source: CouponController.active and CouponService.setActive

Returns

API 53 Request a full order return

POST /api/orders/11111111-1111-4111-8111-111111111111/returns

Access: Bearer access token

Request body

```
{"reason": "The item is damaged"}
```

Success response example HTTP 200

```
{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "DELIVERED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "COLLECTED",
  "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedAt": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
  "checkout": null,
  "createdAt": "2026-09-15T10:00:00Z",
  "expiresAt": null,
  "refund": {"status": "NONE", "razorpayRefundId": null, "failure": null},
  "carrier": null,
  "trackingNumber": null,
  "shippedAt": null,
  "deliveredAt": "2026-09-15T10:00:00Z",
  "subtotal": 2000.0,
  "discount": 200.0,
  "couponCode": "SAVE10",
  "returnRequest": {
    "status": "REQUESTED",
    "reason": "The item is damaged",
    "decisionNote": null,
    "requestedAt": "2026-09-15T10:00:00Z",
    "receivedAt": null,
    "restock": null,
    "manualRefundReference": null
  }
}
```

Behavior and reasons

Owned order, nonblank reason ≤ 1000 , status DELIVERED, and deliveredAt no more than 30 days ago. One full-order request is stored. A repeat with the same trimmed reason is idempotent; a different reason conflicts. The order remains DELIVERED during request/approval; no stock or money is moved yet.

Other responses and failure conditions

404 foreign or missing order; 409 outside window/not delivered/different repeat; 400 validation; 401.

Source: ReturnController.request and OrderLifecycle.requestReturn

API 54 Approve or reject a return

```
PATCH /api/admin/orders/11111111-1111-4111-8111-111111111111/return
```

Access: Bearer token with ROLE_ADMIN

Request body

```
{"decision": "APPROVED", "note": "Return all items for inspection"}
```

Success response example HTTP 200

```
{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "DELIVERED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "COLLECTED",
  "items": [{ "productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2 }],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
  "checkout": null,
  "createdAt": "2026-09-15T10:00:00Z",
  "expiresAt": null,
  "refund": { "status": "NONE", "razorpayRefundId": null, "failure": null },
  "carrier": null,
  "trackingNumber": null,
  "shippedAt": null,
  "deliveredAt": "2026-09-15T10:00:00Z",
  "subtotal": 2000.0,
  "discount": 200.0,
  "couponCode": "SAVE10",
  "returnRequest": {
    "status": "APPROVED",
    "reason": "The item is damaged",
    "decisionNote": "Return all items for inspection",
  }
}
```

```

    "requestedAt": "2026-09-15T10:00:00Z",
    "receivedAt": null,
    "restock": null,
    "manualRefundReference": null
  }
}

```

Behavior and reasons

decision is APPROVED or REJECTED; nonblank note <=1000. Only REQUESTED can change. Identical decision and trimmed note are idempotent. A rejection cannot later be received or changed to approval via this endpoint.

Other responses and failure conditions

409 not REQUESTED or changed repeat; 400 validation; 401/403; 404 missing.

Source: ReturnController.decide and OrderLifecycle.decideReturn

API 55 Record receipt and inventory disposition

```
POST /api/admin/orders/11111111-1111-4111-8111-111111111111/return/receive
```

Access: Bearer token with ROLE_ADMIN

Request body

```
{"restock": false}
```

Success response example HTTP 200

```

{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "RETURNED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "REFUND_PENDING",
  "items": [{ "productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2 }],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  },
  "cartCleared": true,
  "failureReason": null,
  "checkout": null,
  "createdAt": "2026-09-15T10:00:00Z",
  "expiresAt": null,
  "refund": { "status": "MANUAL_REQUIRED", "razorpayRefundId": null, "failure": null },
  "carrier": null,
  "trackingNumber": null,
  "shippedAt": null,
  "deliveredAt": "2026-09-15T10:00:00Z",
  "subtotal": 2000.0,

```

```

"discount": 200.0,
"couponCode": "SAVE10",
"returnRequest": {
  "status": "RECEIVED",
  "reason": "The item is damaged",
  "decisionNote": "Return all items for inspection",
  "requestedAt": "2026-09-15T10:00:00Z",
  "receivedAt": "2026-09-15T10:00:00Z",
  "restock": false,
  "manualRefundReference": null
}
}

```

Behavior and reasons

Require explicit restock Boolean and APPROVED return. Commit RETURN_RECEIVING with the decision before a remote call. Product-service either restocks all quantities or marks them discarded. Then mark RETURNED/RECEIVED and start refund handling. Online uses Razorpay; COD waits for an external manual refund. During outage the API can return 200 with RETURN_RECEIVING because advance defers recovery; 200 alone does not prove physical receipt processing is complete.

Other responses and failure conditions

409 unapproved or changed receipt decision; 400 null restock; 401/403. Recovery handles temporary inventory failure.

Source: ReturnController.receive and OrderLifecycle.startReceiveReturn receiveReturn

API 56 Record a completed COD refund

```
POST /api/admin/orders/11111111-1111-4111-8111-111111111111/refund/manual
```

Access: Bearer token with ROLE_ADMIN

Request body

```
{"reference": "BANK-REFUND-001"}
```

Success response example HTTP 200

```

{
  "id": "11111111-1111-4111-8111-111111111111",
  "status": "RETURNED",
  "paymentMethod": "CASH_ON_DELIVERY",
  "paymentStatus": "REFUNDED",
  "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}],
  "totalPrice": 1800.0,
  "currency": "INR",
  "shippingAddress": {
    "createdAt": "2026-09-15T10:00:00Z",
    "updatedAt": "2026-09-15T10:00:00Z",
    "createdBy": null,
    "updatedBy": null,
    "id": 1,
    "line1": "10 Example Road",
    "line2": "Apartment 2",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411001",
    "country": "India",
    "defaultAddress": true
  }
}

```

```

    },
    "cartCleared": true,
    "failureReason": null,
    "checkout": null,
    "createdAt": "2026-09-15T10:00:00Z",
    "expiresAt": null,
    "refund": {"status": "PROCESSED", "razorpayRefundId": null, "failure": null},
    "carrier": null,
    "trackingNumber": null,
    "shippedAt": null,
    "deliveredAt": "2026-09-15T10:00:00Z",
    "subtotal": 2000.0,
    "discount": 200.0,
    "couponCode": "SAVE10",
    "returnRequest": {
      "status": "RECEIVED",
      "reason": "The item is damaged",
      "decisionNote": "Return all items for inspection",
      "requestedAt": "2026-09-15T10:00:00Z",
      "receivedAt": "2026-09-15T10:00:00Z",
      "restock": false,
      "manualRefundReference": "BANK-REFUND-001"
    }
  }
}

```

Behavior and reasons

Nonblank reference <=150. Require RETURNED, CASH_ON_DELIVERY and MANUAL_REQUIRED. Mark the already externally paid refund processed. A repeat with the same trimmed reference succeeds. This endpoint neither transfers money nor independently verifies a bank payment; the admin attestation is trusted.

Other responses and failure conditions

409 wrong state/method or changed processed reference; 400 validation; 401/403.

Source: ReturnController.manual and OrderLifecycle.manualRefund

Notification service

API 57 Read your notification inbox

GET /api/notifications?page=0&size=20

Access: Bearer access token

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{
  "content": [
    {
      "eventId": "22222222-2222-4222-8222-222222222222",
      "userId": 1,
      "orderId": "11111111-1111-4111-8111-111111111111",
      "eventSequence": 1,
      "type": "OrderConfirmed",
      "message": "Your order is confirmed.",
      "occurredAt": "2026-09-15T10:00:00Z",
      "receivedAt": "2026-09-15T10:00:00Z"
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 20,
    "sort": {"sorted": false, "unsorted": true, "empty": true},
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "last": true,
  "totalPages": 1,
  "totalElements": 1,
  "size": 20,
  "number": 0,
  "sort": {"sorted": false, "unsorted": true, "empty": true},
  "first": true,
  "numberOfElements": 1,
  "empty": false
}
```

Behavior and reasons

Filter by authenticated numeric user ID, newest receivedAt first. page >=0, size 1 to 100. Each notification is stored once by eventId. This API does not mark items read or delete them. Received order and return events drive the inbox; user-events are a separate stream.

Other responses and failure conditions

401 invalid/missing JWT; invalid pagination.

Source: NotificationController.list and NotificationRepository

Internal APIs

API 58 Fetch a checkout cart snapshot

```
GET /internal/carts/1
```

Access: X-Service-Key on cart-service port 8084

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 200

```
{"userId": 1, "version": 3, "items": {"1": 2}}
```

Behavior and reasons

Path template /internal/carts/{userId}. No explicit public gateway route. Return only stable quantities and JPA version, not client prices. Missing/empty cart returns 409. JSON object keys are strings even though the Java map uses Long.

Other responses and failure conditions

401 wrong/missing service key; 409 empty cart.

Source: CheckoutCartController.snapshot

API 59 Consume only a matching cart version

```
POST /internal/carts/1/consume
```

Access: X-Service-Key on cart-service port 8084

Request body

```
{"version": 3}
```

Success response example HTTP 200

```
{"cleared": true}
```

Behavior and reasons

Lock the cart. Clear only when cart exists and a non-null requested version equals the current version. Missing/stale/null version returns 200 {"cleared":false}; preserve items added after checkout. A repeated successful consumption can return false after the version changed, so effect idempotency does not mean identical response bytes.

Other responses and failure conditions

401 wrong/missing key; malformed JSON binding failure.

Source: CheckoutCartController.consume

API 60 Reserve stock for an order

```
PUT /internal/inventory/reservations/11111111-1111-4111-8111-111111111111
```

Access: X-Service-Key on product-service port 8083

Request body

```
{"userId": 1, "items": {"1": 2}}
```

Success response example HTTP 200

```
{
  "id": "11111111-1111-4111-8111-111111111111",
  "state": "RESERVED",
  "items": [{"productId": 1, "name": "Everyday Phone", "price": 1000.0, "quantity": 2}]
}
```

Behavior and reasons

UUID order ID; positive userId; nonempty map with <=100 entries; positive IDs and quantities. Existing reservation must match user and quantities and still be RESERVED. For a new reservation lock products in ascending ID order, require active/enough stock, snapshot prices/names and subtract stock in one local transaction. Any invalid line rolls back all deductions.

Other responses and failure conditions

422 product missing/inactive/insufficient; 409 different or already released reservation; 400 validation; 401 key.

Source: InventoryController.reserve and InventoryService.reserve

API 61 Release reserved stock

```
DELETE /internal/inventory/reservations/11111111-1111-4111-8111-111111111111
```

Access: X-Service-Key on product-service port 8083

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

A repeated RELEASED reservation is a no-op. Unknown order IDs create a released tombstone so a delayed reserve cannot recreate stock demand after cancellation. RESERVED quantities are restored with overflow checking. COMMITTED and returned states cannot be released.

Other responses and failure conditions

409 committed/returned or missing reserved product or integer overflow; 401 key.

Source: InventoryController.release and InventoryService.release

API 62 Commit shipped inventory

```
POST /internal/inventory/reservations/11111111-1111-4111-8111-111111111111/commit
```

Access: X-Service-Key on product-service port 8083

Request body

No request body. Parameters, when present, are shown in the URL or headers above.

Success response example HTTP 204

Empty response body.

Behavior and reasons

A reservation must exist and not be RELEASED. Mark COMMITTED so cancellation cannot restore stock already shipped. Repeats are safe; returned states are preserved rather than rewritten to COMMITTED.

Other responses and failure conditions

404 unknown reservation; 409 released reservation; 401 key.

Source: InventoryController.commit and InventoryService.commit

API 63 Apply the return stock decision

```
POST /internal/inventory/reservations/11111111-1111-4111-8111-111111111111/return
```

Access: X-Service-Key on product-service port 8083

Request body

```
{"restock": true}
```

Success response example HTTP 204

Empty response body.

Behavior and reasons

Explicit restock Boolean. RESERVED or COMMITTED can transition to RETURNED (add quantities once) or RETURNED_DISCARDED (no addition). Same target repeats are no-ops. Opposite disposition conflicts. Product rows are locked in sorted order; missing products and overflow fail. This endpoint trusts the internal caller to have enforced delivered-order approval and receipt; it cannot verify orderdb itself.

Other responses and failure conditions

404 unknown reservation; 409 released/already returned differently/missing restock product/overflow; 400 null restock; 401 key.

Source: InventoryController.receiveReturn and InventoryService.receiveReturn

Infrastructure and provider interfaces

Razorpay checkout response variant

Order responses use the same OrderView structure shown above. In a usable online checkout, paymentMethod is RAZORPAY, status is PENDING_PAYMENT, paymentStatus is UNPAID, and checkout contains the object below. The amount is an integer number of paise. The browser receives the public test key ID; the secret key stays on the server. Other order fields remain present, including expiry and refund state.

```
{
  "keyId": "rzp_test_EXAMPLE",
  "razorpayOrderId": "order_demo1",
  "amount": 180000,
  "currency": "INR",
  "testMode": true
}
```

Outbound provider requests

The following seven HTTP operations are emitted by RazorpayClient in this repository. This is a source contract description, not a claim to exhaustively document the current external provider API. Base URL: <https://api.razorpay.com/v1>. Every call uses HTTP Basic authentication with configured key ID and secret. Never embed the secret in browser JavaScript or this document. Responses below include the fields consumed by this application; Razorpay can return additional fields.

POST /orders

Request JSON

```
{
  "receipt": "11111111-1111-4111-8111-111111111111",
  "amount": 180000,
  "currency": "INR",
  "partial_payment": false
}
```

Successful response consumed fields

```
{
  "id": "order_demo1",
  "receipt": "11111111-1111-4111-8111-111111111111",
  "amount": 180000,
  "currency": "INR",
  "status": "created"
}
```

GET /orders?receipt=11111111-1111-4111-8111-111111111111&count=100

Request body: none.

Successful response consumed fields

```
{
  "items": [
    {
      "id": "order_demo1",
      "receipt": "11111111-1111-4111-8111-111111111111",
      "amount": 180000,

```

```

    "currency": "INR",
    "status": "paid"
  }
]
}

```

GET /orders/order_demo1

Request body: none.

Successful response consumed fields

```

{
  "id": "order_demo1",
  "receipt": "11111111-1111-4111-8111-111111111111",
  "amount": 180000,
  "currency": "INR",
  "status": "paid"
}

```

GET /orders/order_demo1/payments

Request body: none.

Successful response consumed fields

```

{
  "items": [
    {
      "id": "pay_demo1",
      "order_id": "order_demo1",
      "amount": 180000,
      "currency": "INR",
      "status": "captured"
    }
  ]
}

```

GET /payments/pay_demo1

Request body: none.

Successful response consumed fields

```

{
  "id": "pay_demo1",
  "order_id": "order_demo1",
  "amount": 180000,
  "currency": "INR",
  "status": "captured"
}

```

POST /payments/pay_demo1/refund

Request JSON

```

{"amount": 180000, "speed": "normal", "receipt": "11111111-1111-4111-8111-111111111111"}

```

Additional header: X-Refund-Idempotency: <persisted refund attempt UUID>. The receipt example represents the attempt ID, not the order ID; these are distinct values in a real request.

Successful response consumed fields

```
{
  "id": "rfnd_demo1",
  "payment_id": "pay_demo1",
  "amount": 180000,
  "currency": "INR",
  "status": "processed"
}
```

GET /refunds/rfnd_demo1

Request body: none.

Successful response consumed fields

```
{
  "id": "rfnd_demo1",
  "payment_id": "pay_demo1",
  "amount": 180000,
  "currency": "INR",
  "status": "processed"
}
```

Framework and static HTTP surfaces

Surface	Request and response	Scope
Gateway storefront	GET / and /index.html → HTML; referenced JS and CSS assets → text resources	Browser interface not JSON business APIs
Actuator	GET /actuator/health → 200 {"status":"UP"}; unhealthy status can be 503	Container management port 9090; exposure and health details depend on profile
Actuator information	GET /actuator/info → configured information object, possibly {}	Container exposes health and info; IDE gateway configuration is broader
Config Server	GET /order-service/default → Environment JSON	name profiles label version state and propertySources; property values omitted here to avoid publishing configuration secrets
Eureka registry	GET /eureka/apps with Accept application/json → applications object	Framework registration heartbeat and deregistration support discovery; not customer commerce operations
HTTP protocol methods	HEAD on supported GET routes omits body; OPTIONS is framework handled	CORS and security policy depend on configuration; no custom business body

Config Server can also expose the framework YAML and properties representations such as /order-service-default.yml. Eureka clients register instance metadata, renew leases and unregister through its framework protocol. These automatically supplied endpoints are distinct from the complete application controller catalog. Do not expose registry, configuration or internal inventory endpoints to customers merely because discovery can resolve their service IDs.

Event request and response equivalents

Kafka messages have no synchronous HTTP response. Producer acknowledgement, consumer acknowledgement and a persisted inbox row occur at separate times. The order outbox publishes an envelope like this, keyed by orderId. eventId deduplicates delivery and sequence identifies order-local progression.

```
{
  "schemaVersion": 1,
  "eventId": "22222222-2222-4222-8222-222222222222",
  "orderId": "11111111-1111-4111-8111-111111111111",
  "userId": 1,
  "sequence": 1,
  "type": "OrderConfirmed",
  "occurredAt": "2026-09-15T10:00:00Z",
  "status": "CONFIRMED",
  "paymentStatus": "UNPAID",
  "refundStatus": "NONE",
  "carrier": null,
  "trackingNumber": null
}
```

Recognized order event types include OrderConfirmed, OrderFailed, PaymentPending, PaymentCaptured, PaymentCollected, OrderExpired, OrderCancelled, RefundRequested, RefundPending, RefundProcessed, RefundFailed, OrderProcessing, OrderShipped, OrderDelivered, ReturnRequested, ReturnApproved, ReturnRejected and ReturnReceived. The consumer validates schema version, UUID identifiers, positive userId and sequence. Unknown types are ignored. A duplicate eventId does not create a second inbox row.

```
{
  "type": "PasswordResetOtpRequested",
  "userId": 1,
  "email": "asha@example.com",
  "occurredAt": "2026-09-15T10:00:00Z"
}
```

User events carry metadata and do not contain the OTP. They are published directly rather than through the order outbox. This sample therefore does not imply functioning email delivery or equivalent crash durability.

Conditions and why they exist

Identity and account guard conditions

The JWT signature check prevents modified `userId` or `role` claims from being accepted. Expiry limits the lifetime of a stolen token. `Product/order/cart/notification` filters check essential claims before building Authentication; `user-service` uses a less strict custom parser/filter and catches token errors. The project shares an HMAC secret, which is convenient for learning but allows any service holding it to mint tokens. Asymmetric signing and centralized key rotation would reduce that trust.

Explicit ownership checks prevent horizontal privilege escalation. `OrderService.owned` and `ShoppingService.review` return 404 for foreign orders. `AccountSettingsService.owned` also verifies address ownership but wraps failure as `ApiException` and therefore 400. `Wishlist` and `notifications` query with the authenticated ID, not user-supplied ownership fields. `UserController.get` by arbitrary ID is an exception: it lacks an ownership/admin check and should be described as a security gap.

`ROLE_ADMIN` is required on product writes, admin catalog and admin `order/coupon/return` operations. The product security matcher checks `/admin/**` before public GET patterns; otherwise a broad public rule could accidentally expose hidden catalog entries. `@EnableMethodSecurity` makes `order-service` `@PreAuthorize` effective. Hiding the Admin link in JavaScript is never the enforcement mechanism.

`Nonblank/length` checks bound meaningful input and storage. Normalizing email and SKU prevents case variants from bypassing logical uniqueness. Database unique constraints remain essential because two requests can both pass an `exists` check before either writes. Application prechecks improve messages; constraints are the final race-resistant enforcement.

The login lock checks apply before password processing. The fifth failure increments the persisted count and sets a deadline; elapsed locks are cleared so a user is not locked forever. `dontRollbackOn` preserves the failed-attempt update. `SecureRandom` OTP generation avoids predictable codes, expiry limits replay time and used flags block reuse of that token. There is still no OTP attempt throttle, hashed token storage, universal access revocation or email-delivery implementation.

Catalog and cart guard conditions

A positive price and nonnegative bounded stock keep ordinary catalog values meaningful. `@Digits` limits scale to two decimals and matches monetary storage. The optional HTTPS image constraint blocks `javascript/data` schemes as product image URLs; the browser also escapes content and accepts HTTPS before rendering an image. It does not guarantee that a remote image host is trustworthy or that the resource is actually an image.

Search rejects negative or inverted price ranges to avoid contradictory requests. Escaping LIKE wildcards makes a user-entered percent sign a literal search character rather than a wildcard expansion. The sort allowlist prevents invalid field names and uncontrolled sort access. Adding `id` as a tie-breaker helps stable offset paging when values are equal; it does not create a snapshot across concurrent inserts. The cap of 100 limits query and payload size.

Cart additions validate the combined quantity, not just the increment, so repeated small additions cannot exceed stock at validation time. Long arithmetic detects overflow before narrowing to `int`. A cart is still not a reservation: another checkout can reduce stock before the customer purchases. Product lookup failures become 503, while a missing product is represented as an unavailable removable cart line on GET.

Cart @Version detects overlapping modifications. An item collection's unique user/product pair blocks duplicate rows. During consumption the exact version match protects a customer who adds another item while payment/checkout is in flight. The browser also includes this version in its idempotency fingerprint so a later identical purchase does not reuse the old order key.

Checkout and inventory guard conditions

Hashing the client idempotency key produces a fixed-length database value; the scope is user ID plus hash so two customers may choose the same textual key independently. The fixed ASCII key format and 8 to 128 length reduce malformed input. Comparing stored address, payment method and coupon prevents a key from silently switching to a different logical checkout.

The second unique constraint on user/cart-version catches different keys racing for one unchanged cart. Persisting CREATING before external work gives recovery a durable identity. Reusing the local UUID as reservation ID makes inventory allocation repeatable even when a response was lost. Map validation checks user match, version presence, nonempty size and positive keys/quantities before any side effects.

Product-service checks every requested product for existence, activity and available quantity. A single local transaction makes a multi-line failure roll back all stock. Sorted product locking reduces cyclic lock acquisition between carts containing the same products in different orders. Pessimistic locking serializes competing reservations; @Version additionally protects entity writes.

An existing reservation must match both user and item map and still be RESERVED. Otherwise the same key could reserve a different basket or resurrect released stock. Release of an unknown ID creates a tombstone, addressing the race where cancellation arrives before the original reserve. RELEASED replay is harmless. COMMITTED/returned states cannot be restored through cancel, because those units have already shipped or been accounted for through returns.

OrderService validates reservation responses instead of trusting a downstream payload blindly. Duplicate line IDs, wrong quantities, mismatched order ID/state or nonpositive prices cause dependency rejection rather than a payment for the wrong basket. Its total is calculated from reserved lines, never from a price field sent by the customer.

Payment and refund guard conditions

The test-key prefix prevents this development integration from charging live Razorpay accounts accidentally. Missing configuration produces 503 instead of an apparently valid online order. The local attempted flag is saved before provider creation; if the network response is ambiguous, lookup by receipt is safer than a blind second POST. This favors duplicate-payment prevention over automatic availability, and some ambiguous cases require manual reconciliation.

The HMAC callback check binds provider order and payment identifiers. Re-querying the provider then binds amount, currency and capture state; both checks are necessary because a signature is not a business-level proof of the expected captured amount. Unique provider order/payment IDs on PurchaseOrder prevent those references from being reused across local orders.

The overdue predicate uses not-before, so equality with an online expiry deadline counts as expired. Expiration applies only to incomplete online stages, not already confirmed fulfillment. closed includes returned orders for refund processing. Captures after cancellation/expiry request a refund without reopening the order. A different payment ID on an order conflicts rather than overwriting the original.

Refund requests use a persisted UUID and the same X-Refund-Idempotency/body across uncertain retries. Provider refund ID, payment ID, amount and INR are revalidated before state changes. PROCESSED is terminal in the local state setter, so an out-of-order pending/failed message cannot regress a completed refund. Confirmed provider failure allows a new attempt, whereas timeouts, conflicts and throttling stay retryable.

External partial refund amounts lead to REVIEW_REQUIRED because this code only accounts for full refunds. That is safer than pretending the unpaid balance is zero. It is not a complete partial-refund or chargeback reconciliation engine. A paid/shipped order affected by an out-of-band refund can require operator review; the ordinary cancellation release path cannot release COMMITTED stock.

Coupon guard conditions

Coupon creation validates bounded values, future expiry, a global usage limit and type. A PERCENT value above 100 is invalid. Normalizing the code makes customer input case-insensitive; code uniqueness still resides in the database. Rule values are immutable through the API, preventing a promotion edit from rewriting past order calculations.

validate before creating an order rejects obviously unusable codes quickly. apply rechecks under a pessimistic coupon lock after the authoritative subtotal is known. The lock and usedCount increment share the order's database transaction, preventing the last permitted use from being spent twice. A quote does not consume a use and its caller-supplied subtotal is never trusted for charging.

The couponRedeemed flag prevents double decrement during repeated cancellation recovery. release is called for CANCELLED, EXPIRED and FAILED, but not a delivered return. The $\text{Math.max}(0, \text{usedCount}-1)$ guard avoids negative counters on inconsistent historical data; it is a safeguard rather than a substitute for reconciling a corrupted counter.

noRollbackFor=CouponRejectedException is narrowly applied to the deterministic rule rejection. It lets OrderService catch that exception and persist the compensating failure status after a remote reservation. It does not suppress database failures or make remote stock part of the same transaction. The order price fields remain snapshots even if a code is later deactivated.

Return and review guard conditions

Return ownership, delivered status and a 30 day window constrain eligibility. A reason is mandatory and bounded. A repeat with identical reason does not create another return, while a changed reason conflicts. Only REQUESTED can receive an admin decision, and only APPROVED can begin receipt. These checks stop accidental skipping from a customer request straight to a refund.

Committing RECEIVING with a fixed restock Boolean before calling inventory permits safe restart. Product-service accepts the same disposition again but rejects the opposite one, so a lost response cannot lead to restocking twice or changing a discard into a later restock. It accepts RESERVED as well as COMMITTED because the order's earlier commit may itself have been delayed. The trusted internal caller enforces delivery/approval; the stock service enforces stock effects.

Manual refund requires a received COD return in MANUAL_REQUIRED and a nonblank reference. Matching completed references are idempotent; other transitions are rejected. This protects workflow consistency but is not evidence from a bank. An administrator remains accountable for the actual external payment.

Reviews require deliveredAt and a product present in the order snapshot. Rating and comment bounds make display and storage predictable. Uniqueness per user/product avoids repeated reviews through multiple orders.

User/order IDs are omitted from the public view. A returned item can still have a legitimate review because it was delivered. Wishlist save checks existence/activity only for new entries; removal always scopes by owner.

Delivery and resilience guard conditions

OrderEvents.MANDATORY refuses publication outside an active transaction, tying outbox insertion to the order update. The publisher waits for Kafka acknowledgment before marking an event published. Deduplication by eventId absorbs at-least-once replay. Schema/version/UUID/user/sequence validation rejects malformed envelopes. Unknown event types are ignored for forward compatibility; malformed known envelopes still require operational handling.

Circuit breakers group by authority, keeping one breaker per downstream instead of one per product or user. A 20-call window with at least 10 calls, 50 percent failure threshold, 30 second open period and three half-open probes limits repeated calls to failing services. HTTP 5xx, 429 and transport/runtime failures count; ordinary business 4xx do not. This avoids opening a breaker merely because customers request unavailable products.

Connect/read timeouts prevent waiting forever. Product/cart/user calls use three second connect and five second read limits; Razorpay reads use eight seconds. Redis connection/read timeouts are 300 milliseconds in product configuration and failures fall back to MySQL. These values are defaults to measure, not universal performance guarantees. Some remote calls occur while database row locks are held, which can amplify contention under dependency latency.

Operations testing and limitations

Starting and configuring the project

The documented IDE order is registry, config server, user/product, cart, order, notification and gateway. MySQL databases are userdb, productdb, cartdb, orderdb and notificationdb. Kafka is normally localhost:9092 for IDE use, with different container-advertised DNS endpoints. The config server native profile reads the local configuration repository; the default uses its configured Git backend. The controller ports are separate from container management port 9090.

Docker Compose provides databases, Kafka, Redis, services and an optional Tempo profile. The core startup script starts services sequentially and waits for health to reduce pressure on an 8 GB development machine. Kubernetes manifests describe single-replica development workloads, persistent volumes, probes and network policies, not a proven high-availability deployment. Container Config Server imports and discovery-locator behavior differ from IDE defaults; always identify the active profile when troubleshooting routing.

Keep JWT_SECRET consistent across JWT-validating services and INTERNAL_SERVICE_KEY consistent between trusted internal callers/receivers. RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET and RAZORPAY_WEBHOOK_SECRET are separate values. The frontend receives the key ID only. Never copy real secrets from .env into documentation or sample requests. The current schema update approach is Hibernate ddl-auto=update; production requires reviewed versioned migrations and backup/restore practice.

Tests and what their results mean

The completed full Maven verification log dated 12 September 2026 records 84 tests with zero failures or errors across all eight services: config server 2, registry 1, gateway 6, user 8, product 23, cart 13, order 30 and notification 1. This is a historical completed build record, not a claim that the running deployment is healthy at any later time.

Product tests exercise filtering, active visibility, admin authorization, cache behavior, seeding, inventory concurrency, tombstones and return dispositions. Cart tests cover ownership, current prices, unavailable products, quantities and versions. Order tests use local HTTP/provider stubs for signature/amount checks, lost responses, duplicate keys, stock failure, refunds, coupons, reviews and returns. Notification tests use embedded Kafka and check deduplication, inbox ownership and return event types.

H2 tests do not prove every MySQL isolation, lock, SQL or migration behavior. Gateway stub tests verify forwarding rather than an entire deployed system. Browser checks used an isolated fixture server and verified UI wiring and responsive layout, not real payment completion. A production-readiness test plan should add real MySQL concurrency, service restart/fault tests, signed provider sandbox callbacks, load tests and migration checks.

Troubleshooting by symptom

If the gateway refuses a TCP connection, no listener is accepting at that address; investigate startup or port configuration before changing controllers. A gateway 503 with services otherwise alive may mean Eureka has no healthy instance or the downstream URI is wrong. Compare explicit routes and the active profile. A 401 usually means missing/invalid token or mismatched JWT secret, while 403 means authentication succeeded without the required role.

An empty cart at checkout returns 409. A second key for an unchanged cart version also returns 409; inspect Orders rather than creating more keys. CREATING/RESERVED can mean a dependency or ambiguous provider

creation is awaiting recovery. Do not manually clear `paymentCreationAttempted` merely to force another provider order. Insufficient stock is a business failure, not a reason to reset a circuit breaker.

A `PENDING_PAYMENT` order needs a captured provider payment, correct amount and matching signature. A late captured payment should become refund work, not fulfillment. `REFUND_FAILED` needs provider inspection before retry; `REVIEW_REQUIRED` is not automatically retried into success. `RETURN_RECEIVING` indicates inventory disposition has not completed. `MANUAL_REQUIRED` means an admin must pay COD money outside the app and record its reference.

Missing OTP email is expected with the current implementation: no delivery adapter exists. Empty notifications after a user registration are not proof of a broken consumer because notification-service consumes order-events, not user-events. Redis failure should fall back to MySQL; if both fail, availability is still lost. Infinite consumer retries can block a poison record and should be visible in logs and consumer lag.

Security and correctness improvements to discuss honestly

The most urgent application-level issue is authenticated cross-user profile reads at `GET /api/users/{id}`. Other hardening items include consistent JWT claim validation, access-token revocation strategy, asymmetric keys, refresh-token locking/reuse detection, token hashing at rest, OTP delivery and throttling, enumeration-resistant password reset, endpoint rate limits and a consistent error model. The current `sessionStorage` design is exposed to JavaScript if XSS occurs; escaping is useful but not a complete CSP/security strategy.

User event publication needs an outbox if registration/settings events must be reliably coupled to database commits. Address-default selection needs concurrency protection. API DTOs should replace exposed persistence entities where contracts need long-term stability. Database indexes should follow query plans, particularly catalog filtering and order/review lookup. The admin stock API could require an optimistic version or ETag to detect stale forms, not only overlapping database transactions.

For distributed recovery, consider leases/skip-locked work claiming, bounded retries with operator queues, dead-letter handling, outbox cleanup and backpressure. Avoid claiming infinite retries guarantee success. Some order operations hold locks while HTTP calls execute; production scaling would benefit from shorter transactions, explicit effect records and carefully designed reconciliation. Keep the existing idempotency invariants while changing orchestration.

There is no taxes/shipping-fee engine, live payment integration, courier integration, partial return/refund engine, review moderation, email provider or recommendation service. The guide describes these as future work rather than claiming they exist. A good interview answer names a concrete next improvement and its failure scenario instead of listing fashionable technologies.

Interview questions and project answers

Answers below describe the checked-in implementation. In an interview, explain the concrete mechanism, then the failure case it addresses, and finally the remaining limitation. Do not claim live payment processing, universal exactly-once delivery or complete access-token revocation.

Question 1 How would you explain the project in two minutes

It is an eight-service Spring ecommerce application with a gateway-served storefront. User, product, cart, order and notification services own their databases; registry and config server support discovery/configuration. The most substantial feature is durable checkout: order-service persists an attempt, reserves stock idempotently, validates Razorpay test payments and recovers failures. I would demonstrate a normal COD order, then a lost-response retry, then a return. I would explicitly say reviews/coupons/wishlists live in order-service and that live payments and courier integrations are not implemented.

Question 2 Why use microservices here

The repository separates business ownership and demonstrates network failure handling, database isolation and event delivery. Inventory can enforce its own stock transaction while order-service controls the broader workflow. The cost is deployment and consistency complexity. For a smaller real product I would consider a modular monolith first; microservices become justified by team boundaries, independent scaling or deployment needs, not simply by the number of entities.

Question 3 What does the API gateway actually do

CatalogRoutes, CartRoutes, OrderRoutes and NotificationRoutes forward the defined paths to lb:// destinations and preserve request headers/bodies. The gateway also serves static storefront files. Backend services enforce JWT and role/ownership rules; the gateway is not a central authentication server. The container profile disables dynamic discovery routes, whereas the workspace config repository enables them. I would inspect the active profile before claiming a path is unreachable.

Question 4 Are all service calls discovered through Eureka

No. Gateway lb:// routes use service discovery, but ProductClient and ServiceClients are ordinary RestClient clients built from configured URLs. Their defaults and container DNS configuration determine the target. This distinction matters during a migration or scaling discussion: a direct base URL needs its own load-balancing or service-DNS strategy.

Question 5 How is a request processed inside a service

The filter validates identity, the controller binds and validates input, the service applies ownership/business/state checks, and repositories run local persistence work. DTOs or selected entities become JSON. For order mutations, TransactionTemplate makes transaction boundaries explicit around persisted progress. Controller validation cannot replace database constraints, and authentication cannot replace authorization.

Question 6 Why separate databases

Each service controls its schema and transactions. A cart quantity does not directly alter product stock, and a product edit cannot rewrite a purchase price. There are no cross-service JPA joins or cross-database foreign keys

enforcing the whole workflow. The tradeoff is duplicated identifiers and eventual consistency, handled through snapshots, internal APIs and compensation.

Question 7 What is the difference between authentication and authorization in this code

JWT parsing establishes who is calling. `ROLE_ADMIN` checks decide whether the caller may mutate products or administer orders. Order ownership and address ownership decide whether a customer may access a specific record. The UI hiding an Admin button is not a security boundary. `GET /api/users/{id}` demonstrates a current authorization gap because it only requires authentication.

Question 8 How are passwords protected

`UserService` hashes passwords through `BCryptPasswordEncoder` and compares using matches. Responses do not expose the hash. Login also tracks failures and imposes a 30 minute lock after five attempts. I would not call this complete account security: rate limits, OTP delivery/throttling, consistent access revocation and audit improvements remain.

Question 9 Why does login avoid rollback for `ApiException`

A wrong password must still increment `failedLoginAttempts` and write `LoginHistory`. If the transaction rolled back when `ApiException` was thrown, five failed requests would never accumulate into a lockout. `dontRollbackOn` preserves that state. It is narrowly useful for this flow; broadly disabling rollback for all failures would risk partial updates.

Question 10 What happens after logout

The specified refresh token is revoked; `logout-all` revokes all refresh rows. The browser clears its session storage. Existing access JWTs are self-contained and still valid until expiry because the filters do not consult a denylist or token version. A production design might use short lifetimes plus token-version checks or centralized introspection for sensitive actions.

Question 11 Does refresh token rotation prevent all replay

It prevents ordinary sequential reuse after the old row is marked revoked. The current service does not lock/version the refresh row or implement token-family reuse detection. Two concurrent requests can race. The frontend shares one refresh promise inside a tab, which reduces accidental concurrency but is not a server-side security guarantee.

Question 12 Why can a user log in before verifying email

`UserService.login` checks `active/account` status and password, not `emailVerified`. Verification currently updates metadata. I would describe this as the implemented policy, then explain how adding an `emailVerified` gate changes registration UX and requires a functioning delivery/resend flow. The project presently publishes OTP-request events without implementing delivery.

Question 13 How does product search work

`ProductSearch` builds a JPA Specification with ANDed optional filters. It lowercases `name/category` matches, escapes LIKE wildcards, validates price ranges, enforces active visibility and allows only specific sort fields. It caps

size at 100 and appends ID for tie-breaking. It is relational contains-search, not Elasticsearch or relevance ranking; a large catalog would need indexes/query analysis or a dedicated search design.

Question 14 Why append ID to sorting

Rows sharing a price or timestamp otherwise have no deterministic relative order. Adding ID gives a stable tie-breaker for paging. Offset paging can still shift during concurrent inserts or updates, so this is not snapshot isolation across requests. Cursor pagination would be a future option for large or rapidly changing lists.

Question 15 Why not cache every product list

The implementation only caches product-by-ID responses. Filter combinations, pagination and active/stock changes make list invalidation more complex. A versioned per-ID key is simple to reason about. The database version read remains, so the cache saves full response loading/serialization rather than eliminating database traffic.

Question 16 What if Redis is down

ProductReadCache catches Redis read/write failures and invokes its database loader. Short Redis timeouts limit delay. Product correctness therefore does not depend on Redis availability. If MySQL is also unavailable, the fallback cannot help. This is graceful degradation, not universal high availability.

Question 17 Does adding to cart reserve stock

No. CartService checks current availability but only stores quantities. Checkout's inventory reservation deducts stock. This prevents abandoned carts from locking inventory indefinitely, but means cart availability is only an estimate and can fail at checkout. The UI and API distinguish estimated totals from the authoritative reserved price.

Question 18 Why use BigDecimal instead of double

Monetary comparisons and refunds require decimal accuracy. Product.price and order amounts use BigDecimal and decimal columns; percent discounts round HALF_UP to two places. Razorpay uses exact integer paise through `movePointRight(2).longValueExact()`. JavaScript display can use numbers, but the server computes the authoritative amount and never accepts client totals as the charge.

Question 19 How do you stop duplicate orders

There are two constraints: unique user/key-hash and unique user/cart-version. The same key reuses an order only when checkout details match; a different key cannot buy the same unchanged cart. A race is settled by the database, not an exists check. The client saves its key before sending and reuses it on an unknown outcome.

Question 20 Why hash an idempotency key

SHA-256 provides a fixed-length representation for storage and indexing. It does not make the operation idempotent by itself; the user-scoped unique constraint and request-identity comparison do that. Keys are case-sensitive. The API bounds their length and allowed characters before the service hashes them.

Question 21 Why include cart version in the browser fingerprint

A later cart can contain exactly the same products and quantities as an earlier purchased cart. If the browser fingerprints only contents/address/method, it might reuse the old key and return the old order. The persisted cart version changes when the collection changes, distinguishing a new purchase from a retry.

Question 22 How does checkout handle an edited cart

It snapshots cart version and quantities. At cleanup, cart-service locks the cart and clears only if the version still matches. If the customer added or changed items during checkout, consume returns `cleared=false` and retains them. The order's `cartCleanupDone` and `cartCleared` flags distinguish a finished cleanup decision from actually clearing the cart.

Question 23 How do you prevent overselling

`InventoryService` locks product rows pessimistically and deducts stock in one `productdb` transaction. It checks active state and available quantity for every line. If any line fails, the transaction rolls back all deductions. The order UUID identifies the reservation so a retry cannot deduct again. Tests include concurrent customers contending for insufficient shared stock.

Question 24 Why acquire stock locks in ascending product ID order

Two carts may list the same products in different input order. If one locks A then B while the other locks B then A, a deadlock is more likely. Sorting IDs makes the acquisition order consistent. It reduces this class of deadlock but does not prove that all database deadlocks are impossible.

Question 25 Why use both optimistic and pessimistic locking

`@Version` detects overlapping entity updates such as cart or product modifications. Pessimistic locks serialize specific critical reads and effects such as stock reservation, order advancement and coupon usage. They solve different timing problems. Long-held pessimistic locks under remote latency can hurt throughput; that is a tradeoff to monitor.

Question 26 What is a released reservation tombstone

When cancellation reaches inventory before reservation, release persists a `RELEASED` row for that order ID. A delayed reserve then finds a released record and fails instead of allocating stock for a closed order. Deleting an unknown reservation without recording anything would leave that race open.

Question 27 Is checkout a distributed transaction

No. Local MySQL transactions do not include remote HTTP effects or Razorpay. The design is an orchestration-style saga/state machine: persist intent, execute idempotent effects, persist results and compensate with release/refund when necessary. A recovery worker retries durable intermediate states after crashes. There is no XA or two-phase commit.

Question 28 Why use TransactionTemplate in order service

It makes the local transaction boundaries around individual state-machine steps explicit. The code commits intent before certain provider calls and uses row locks during transitions. A single long `@Transactional` checkout

would still not atomically include remote services and would increase contention. Some current steps still hold locks during HTTP calls, so shortening those is a future refinement.

Question 29 What if inventory reserves stock but the response is lost

The local order can remain CREATING. Recovery retries with the same reservation UUID and quantities. Product-service finds the existing matching RESERVED record and returns the original snapshot without another deduction. OrderService validates that response and continues. This is effect-level idempotency across an ambiguous network outcome.

Question 30 What if Razorpay creates an order but the response is lost

paymentCreationAttempted is already persisted. Recovery searches by the local UUID receipt and validates the matching provider order. It does not blindly post another create. If no unambiguous match is found, the order can remain awaiting reconciliation. The design deliberately sacrifices automatic progress in that case to avoid duplicate payment orders.

Question 31 Why are payment signature checks not enough

The signature authenticates callback identifiers, but the application must also verify that the payment is captured for the correct provider order, exact expected amount and INR currency. OrderService reads the provider and compares those values. A valid callback about another amount or uncaptured authorization must not trigger fulfillment.

Question 32 How do webhooks differ from browser callbacks

The browser callback uses the payment signature over stored order ID and payment ID, and requires the customer's authenticated order ownership. The webhook uses the configured webhook secret over the raw HTTP body and no customer JWT. Both lead to provider-side reconciliation rather than trusting supplied status alone. Recovery provides a third route if either callback is missed.

Question 33 Why validate the raw webhook body

HMAC is defined over exact bytes. Parsing and reserializing JSON can change whitespace, field order or encoding and invalidate the signature even when the logical JSON is similar. PaymentWebhookController accepts byte[] so RazorpayClient can verify before parsing.

Question 34 What happens to a payment captured after expiry

The order stays closed, its stock is not re-reserved, and the captured money becomes refund work. This prevents selling stock that may already have been released and bought by another customer. Money state and fulfillment eligibility are independent; a successful charge does not necessarily mean the business order can reopen.

Question 35 Why keep order payment and refund states separate

A cancelled order can still need money returned, and a delivered COD order uses COLLECTED rather than PAID. Separate dimensions represent these realities without an explosion of combined enum values. Service methods constrain valid transitions, while OrderView exposes all dimensions so clients can present an accurate state.

Question 36 How are refund retries protected

A refund attempt UUID is persisted and sent in X-Refund-Idempotency with an identical body across uncertain retries. Responses are checked for correct provider IDs, amount and currency. A new attempt is allowed only after confirmed terminal failure; pending or unknown outcomes keep the original identity. This relies on the provider contract and local state discipline, not a magical global exactly-once guarantee.

Question 37 What does REVIEW REQUIRED mean

The code detected an external partial refund instead of the full expected amount. It does not implement partial refund allocation across lines or discounts, so it flags operator reconciliation. I would avoid claiming that repeatedly pressing retry solves this state; a partial-refund model and provider reconciliation are needed.

Question 38 How are coupon limits concurrency safe

CouponService.apply locks the coupon row while holding the order transaction. It rechecks expiry, minimum spend and usedCount, then increments usage and snapshots the discount atomically with the order's pricing state. Two contenders for the last use serialize on that lock. A quote never increments usage.

Question 39 Why validate a coupon twice

The early check rejects unusable input before expensive work. The later locked check uses the authoritative reserved subtotal and catches a coupon that expired, was disabled or exhausted in the meantime. Only the latter authorizes the applied discount. If it fails after reservation, the order records failure and compensates inventory.

Question 40 Why noRollbackFor on coupon rejection

A deterministic coupon rejection after a remote stock reservation should let OrderService persist CANCELLING/FAILED intent. If the participating coupon method marked the whole transaction rollback-only, that intent could disappear while stock remained reserved. The narrow exception rule preserves the compensation path; it does not ignore infrastructure/database errors.

Question 41 What does the one rupee discount cap actually guarantee

For subtotal ≥ 1 it caps discount at subtotal-1, leaving one rupee payable. For subtotal < 1 , $\max(\text{subtotal}-1, 0)$ is zero, so it applies no discount but does not raise the amount. I would quote the formula precisely rather than repeat the comment as an absolute invariant. Provider minimum amounts need validation before live use.

Question 42 Why do returned orders keep coupon usage consumed

The current promotion policy counts a completed purchase even if later returned. Failed, cancelled and expired orders release use; delivered/returned orders do not. That prevents repeatedly buying and returning to recycle a limited promotion. It is a business-policy choice, so changing it requires explicit rules and idempotent accounting.

Question 43 How do you make returns idempotent

The request is one per order with a fixed reason, the decision is recorded, and receipt persists a fixed restock Boolean before invoking inventory. Product-service records RETURNED or RETURNED_DISCARDED and accepts

only a repeat of the same target. The order and refund identities then prevent duplicate money effects. Unknown outcomes are retried with the same identities.

Question 44 Why does inventory return accept RESERVED as well as COMMITTED

A prior shipment commit can be delayed even though the order progressed to delivered. The order coordinator is trusted to enforce ownership, delivery, approval and receipt. Inventory focuses on one-time stock disposition and can handle either allocated state. Its internal service key is a coarse trust boundary; per-service identities would strengthen it.

Question 45 Can the application refund COD automatically

No. After approved receipt it marks MANUAL_REQUIRED. An admin performs an external payment and records a reference, which sets REFUNDED/PROCESSED locally. The system does not call a bank API or independently verify the reference. I would demonstrate the state transitions without pretending this endpoint transfers funds.

Question 46 What makes a review a verified purchase

The backend checks the authenticated order owner, a non-null delivery timestamp and the reviewed product in the immutable item snapshot. A database unique user/product pair limits one review. The public DTO omits personal and order identifiers. It is not based on a checkbox or a caller-supplied verifiedPurchase flag.

Question 47 Is wishlist move to cart atomic

No. store.js reads the cart, adds one unit only if absent, and deletes the saved entry after success. If the add response is lost, a retry can see the item already in the cart and avoid adding again. Multiple tabs can still race, and there is no single cross-service transaction. A dedicated idempotent move operation would be a future improvement if strict semantics were needed.

Question 48 Why use an outbox

Saving an order and publishing Kafka directly creates a gap if the process crashes between the two. OrderEvents stores an outbox record in the same transaction as order state. The publisher later delivers and marks it. This guarantees durable intent to publish for committed order changes, while duplicate delivery remains possible and must be handled by consumers.

Question 49 Does the project provide exactly once delivery

No global exactly-once guarantee is implemented. A Kafka send can succeed before publishedAt commits, so the publisher may resend. The consumer uses eventId as the primary identity to make the inbox effect repeat-safe. I would describe this as at-least-once delivery with deduplicated processing, and state that external provider effects have their own idempotency contracts.

Question 50 Why use order ID as the Kafka key

Records for the same order are directed consistently to a partition for a fixed topic partitioning setup, helping preserve their broker order. eventSequence also records logical order progression. Across different orders there is no total ordering requirement. The inbox is sorted by receivedAt, which is not an explicit guarantee of global business-event sequence.

Question 51 What happens to a poison Kafka message

The consumer's error handler retries exceptions indefinitely every five seconds and does not silently discard failures. This preserves persistence attempts but can stall that partition on permanently malformed data. Unknown event types are intentionally ignored, which is a different case. A production improvement is a retry/dead-letter policy with alerts and audited replay.

Question 52 Do user events have the same reliability as order events

No. User EventPublisher directly invokes KafkaTemplate and has no transactional outbox. Its database/event dual-write gap differs from OrderEvents. The OTP event payload contains metadata but no OTP, and no delivery adapter consumes it here. I would propose applying the outbox pattern only after defining what delivery/recovery guarantee those user workflows need.

Question 53 How does the circuit breaker distinguish outages from business errors

The RestClient interceptor counts transport/runtime errors, 5xx and 429 as failures. Ordinary business 4xx count as successful downstream communication. This avoids opening the circuit due to missing products or invalid customer requests. The breaker groups by downstream authority with a 20-call window, minimum 10 calls and 50 percent threshold.

Question 54 How is a retry different from a circuit breaker

A retry attempts work again; a breaker temporarily refuses calls to a failing dependency. This project's scheduled order recovery drives retries of durable business work, while the circuit breaker limits immediate outgoing pressure. It does not automatically retry payment POSTs, which could be unsafe without the persisted attempt rules.

Question 55 What happens when a service restarts mid checkout

The worker queries persisted intermediate orders and resumes advance. Inventory IDs, provider receipt/attempt markers and progress flags determine which effects can be replayed safely. This works only for recorded progress and modeled failures; corrupt data, missing credentials or ambiguous provider state can still require an operator. A restart test should inject faults between every persisted marker and remote effect.

Question 56 What do the tests prove and not prove

The completed build recorded 84 passing tests, with H2, local HTTP stubs and embedded Kafka for selected behaviors. They prove those modeled contracts and regressions under the test environment. They do not prove MySQL production lock behavior, cloud networking, load capacity or real live payment correctness. I would add database-specific and fault-injection tests rather than using the pass count as a production readiness claim.

Question 57 What is the biggest security fix you would make first

Restrict or remove GET /api/users/{id}, because it currently exposes another user's profile including email/phone to any authenticated caller. Then address revocation and refresh concurrency, OTP delivery/throttling, secret/key management and rate limiting. A concrete code-path defect is a better answer than saying only that HTTPS should be enabled.

Question 58 How would you scale order recovery

The current worker selects bounded batches and uses row locks/progress checks, but multiple instances can revisit the same candidates and long HTTP calls can hold locks. I would introduce explicit work claiming or leases, skip-locked access where appropriate, exponential backoff and separate reconciliation states. Every change must preserve one reservation/order/refund identity and safe recovery after lease expiry.

Question 59 How would you implement partial returns

I would add return lines with requested/approved/received quantities, per-line refund allocation including discounts, cumulative refunded/restocked constraints and separate idempotency identities for receipts/refunds. The current whole-order state and total refund cannot simply be reused by changing one quantity. Provider reconciliation, rounding and previously refunded amounts must be modeled explicitly.

Question 60 What would you improve before a production launch

I would prioritize authorization gaps and token lifecycle, real OTP delivery, reviewed schema migrations, database-backed concurrency/failure tests, TLS and managed secrets, backups and observability. Then refine work claiming, outbox/dead-letter operations and shorter transactions. Live payments, tax/shipping and courier integrations would be introduced with explicit contracts and sandbox verification. I would present these as a prioritized plan, not as already implemented features.

Source map and practical walkthrough

Paths below are relative to D:\spring projects\ecommerce. Class names in API entries can be located under these roots. This source map makes the handbook reviewable against the implementation rather than treating examples as a separate API specification.

User controllers and account logic

user-service/user-service/src/main/java/com/ecommerce/user_service

Catalog cache and stock reservations

product-service/src/main/java/com/ecommerce/product_service

Versioned cart and checkout snapshot

cart-service/src/main/java/com/ecommerce/cart_service

Checkout payment and recovery coordinator

order-service/src/main/java/com/ecommerce/order_service

Notification ingestion and inbox

notification-service/src/main/java/com/ecommerce/notification_service

Browser and gateway

api-gateway/src/main/resources

Central service properties

ecommerce-config-repo

Controller coverage register

Controller source	Handler methods
CartController.java	5
CheckoutCartController.java	2
NotificationController.java	1
AdminOrderController.java	4
OrderController.java	5
PaymentWebhookController.java	1
ReturnController.java	4
CouponController.java	4
ShoppingController.java	5
ProductController.java	6

Controller source	Handler methods
InventoryController.java	4
AddressController.java	5
AuthController.java	5
HealthController.java	1
LegacyPasswordController.java	1
PreferencesController.java	2
SessionController.java	2
UserController.java	7

Compatibility aliases can map more than one URL to one handler. Framework endpoints, static files and outbound provider requests are documented separately. The 63 reference entries cover 64 handler methods because the legacy password reset handler shares one entry with the equivalent authentication handler. Aliases and concrete product or order IDs do not require duplicated entries.

A reproducible demonstration sequence

1. Start infrastructure and services with the repository run configuration. Confirm database connectivity, Eureka registrations and service health. Configure placeholder secrets with local test values using the documented environment setup; do not commit real credentials.
2. Use an existing authorized admin account to create an active product with sufficient stock and optionally SAVE10. There is no public API that grants administrator privileges.
3. Register a customer, log in, retain both tokens and add an owned delivery address. Browse the public catalog and add two units of the product to the cart.
4. Submit checkout with a fresh Idempotency-Key. Repeat the exact request with the same key and verify the same order ID. Change a key-bound field and verify a conflict. Add to the cart after the snapshot and verify checkout cleanup preserves the newer cart.
5. For COD, advance PROCESSING then SHIPPED with carrier and tracking, then DELIVERED with cashCollected=true. For online checkout, use the configured Razorpay test flow and verify the captured payment before fulfillment.
6. Create a verified-purchase review, request a return, approve it and record physical receipt with an explicit restock decision. Confirm inventory is restored at most once. COD requires an externally paid refund reference; online refunds can remain pending until provider reconciliation.
7. Read the notification inbox and inspect order event persistence. Repeat selected requests and simulate a temporary dependency outage to explain recovery. Use test data because these operations change orders and stock.

Evidence and maintenance

The prior full verification log dated 12 September 2026 records successful Maven verification across all eight modules and 84 tests. That is historical test evidence, not a new execution or a guarantee that a running deployment is healthy on 15 September. Update this guide when DTO fields, mappings, enum values, security matchers, provider validation, migrations or state transitions change. The build inputs in docs/handbook-build preserve the example catalog and narrative for regeneration.